

分类号

学校代码 10487

学号 M202076675

密级

华中科技大学

硕士学位论文

(学术型 ☐ 专业型 ☒)

基于深度神经网络的个性化实时音乐 推荐系统研究与实现

学位申请人：杨率

学科专业：软件工程

指导教师：卢力 副教授

答辩日期：2023年5月15日

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Professional Master Degree**

**Research and Implementation of Personalized Real-time
Music Recommendation System Based on Deep Neural
Network**

Candidate : YANG Shuai

Major : Software Engineering

Supervisor : Assoc. Prof. LU Li

Huazhong University of Science and Technology

Wuhan 430074, P. R. China

May, 2023

摘要

随着互联网技术的蓬勃发展,网络信息量爆炸式增长,信息过载已然成为一个不可避免的问题。推荐系统作为解决信息过载的方式,面临着许多挑战,比如,推荐系统暴露出的个性化特征提取不足、实时性低等问题。

针对上述问题,鉴于深度神经网络在各领域表现出的优异性能和效果,结合深度学习中长短期记忆网络和卷积神经网络的特性,提出了一种可以感知用户在不同时间序列下喜好信息的个性化音乐推荐算法(CNN-LSTM-SA)。此推荐算法利用长短期记忆网络适合处理时序特征的特点,从用户兴趣序列信息中学习用户喜好;利用卷积神经网络适合处理高维数据的特点,从非序列高维稀疏信息中学习用户和音乐之间的偏好关系;使用注意力机制增强模型对用户兴趣序列中不同信息重要性的理解效果。针对异构数据的不同处理,提升了个性化特征的提取能力,进而完成音乐的个性化实时推荐。基于 CNN-LSTM-SA 推荐算法,以提高系统运行性能和实时性为目标,完成了音乐推荐系统的需求分析与整体架构设计,并选用 Flink 实时计算框架以及 Java EE 开发框架工具 Spring Cloud、MySQL、Kafka、Filebeat、Redis、Elasticsearch 等组件对各核心功能模块进行了详细设计与实现,系统由音乐推荐和搜索模块、数据可视化和交互模块、用户和系统管理模块组成。在系统中,基于 CNN-LSTM-SA 推荐算法构建了具备个性化实时推荐能力和离线推荐能力的音乐推荐模块;基于 B/S 架构实现了交互性良好的系统可视化和异常告警能力;使用 Flink 实时计算框架、Kafka 消息队列和 Redis 缓存提升了实时数据处理能力和实时推荐能力。

实验结果表明,CNN-LSTM-SA 算法在 Top 50 命中率上达到了 86.3%,解决了个性化特征提取不足的问题,满足个性化推荐的要求。基于 CNN-LSTM-SA 推荐算法的个性化实时音乐推荐系统在系统功能测试表现良好,在系统性能测试上延迟可控制在秒级,解决了实时性低的问题,符合实时推荐系统的设计要求,具有较好的实际意义和实用价值。

关键词: 实时推荐系统; 个性化推荐算法; 长短期记忆网络; 卷积神经网络; Flink

Abstract

With the booming development of Internet technology and the explosive growth of online information, information overload has become an inevitable problem. Recommendation systems, as a way to solve information overload, face many challenges, such as insufficient extraction of personalized features and low real-time performance exposed by recommendation systems.

To address these problems, in view of the excellent performance and effectiveness of deep neural networks in various fields, a personalized music recommendation algorithm (CNN-LSTM-SA) that can sense user preference information under different time sequences is proposed by combining the characteristics of long and short-term memory networks and convolutional neural networks in deep learning. This recommendation algorithm uses the characteristics of long short-term memory network suitable for processing temporal sequence features to learn user preferences from user interest sequence information; uses the characteristics of convolutional neural network suitable for processing high-dimensional data to learn the preference relationship between users and music from non-sequential high-dimensional sparse information; uses attention mechanism to enhance the model's understanding effect on the importance of different information in user interest sequence. The different processing for heterogeneous data improves the extraction of personalized features, and then completes the personalized real-time recommendation of music. Based on the CNN-LSTM-SA recommendation algorithm, the requirement analysis and overall architecture design of the music recommendation system are completed with the goal of improving system operation performance and real-time performance, and the Flink real-time computing framework and Java EE development framework tools such as Spring Cloud, MySQL, Kafka, Filebeat, Redis, and The system consists of a music recommendation and search module, a data visualization and interaction module, and a user and system management module. In the system, a music recommendation module with personalized real-time recommendation capability and offline recommendation capability is built based on CNN-LSTM-SA recommendation algorithm; the system visualization and exception alert capability with good interactivity are realized based on B/S architecture; the real-time data processing capability and real-time recommendation capability are enhanced

by using Flink real-time computing framework, Kafka message queue and Redis cache.

The experimental results show that the CNN-LSTM-SA algorithm achieves 86.3% in the Top 50 hit rate, solving the problem of insufficient personalized feature extraction and meeting the requirements of personalized recommendation. The personalized real-time music recommendation system based on CNN-LSTM-SA recommendation algorithm performs well in the system function test, and the delay can be controlled at the second level in the system performance test, which solves the problem of low real-time performance and meets the design requirements of real-time recommendation system and has good practical significance and value.

Key words: Real-time recommendation systems, personalized recommendation algorithms, LSTM, CNN, Flink

目 录

1 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究与发展现状.....	2
1.3 主要研究内容.....	8
2 相关技术与理论	11
2.1 协同过滤算法.....	11
2.2 神经网络算法.....	14
2.3 词嵌入技术.....	18
2.4 本章小结.....	20
3 个性化实时音乐推荐算法研究	21
3.1 问题描述.....	21
3.2 CNN-LSTM-SA 推荐算法设计	22
3.3 CNN-LSTM-SA 算法训练过程.....	26
3.4 CNN-LSTM-SA 算法实验及分析.....	28
3.5 本章小结.....	36
4 个性化实时音乐推荐系统分析与设计	37
4.1 音乐推荐系统需求分析.....	37
4.2 音乐推荐系统架构设计.....	41
4.3 音乐推荐系统功能设计.....	44
4.4 音乐推荐系统数据库设计	50
4.5 本章小结.....	54
5 个性化实时音乐推荐系统实现	55
5.1 音乐推荐系统开发和部署环境	55
5.2 音乐推荐系统功能实现.....	56
5.3 本章小结.....	62
6 个性化实时音乐推荐系统测试	63
6.1 音乐推荐系统测试目的和方法	63
6.2 音乐推荐系统测试环境.....	63
6.3 音乐推荐系统测试内容与结果分析	64
6.4 本章小结.....	67
7 总结与展望.....	68
7.1 全文总结.....	68
7.2 研究展望.....	69
参考文献.....	70

1 绪论

1.1 研究背景与意义

随着互联网生态的完善和 4G、5G 通信技术的普及，互联网更加频繁出现在日常生活中。根据《中国互联网络发展状况统计报告》第 50 次报告，截至 2022 年 6 月，我国网民规模高达 10.51 亿，网络音乐用户规模达 6.58 亿，在线音乐行业用户规模持续增长^[1]。在大规模互联网现实下，当消费者面临大量场景和多样化的选择时，会导致消费者无法选择适合自身的信息，出现信息过载现象^[2,3]。在这样背景下，帮助用户降低信息过载风险具有十分广泛的研究意义。

对于上述的信息过载现象，已经有非常多优秀的技术专家和学者提出了相应的应对办法，可简单的分为以下三类：信息分类、搜索引擎、推荐系统^[4]。

作为信息分类方式之一的门户网站是互联网诞生之初最早的产物，它简单地将人们常用的热门网站通过分类的方式聚合在一起，用户根据已有的使用经验，选择对应分类的网站。这种方式以最简单的实现减少了人们在海量互联网信息中找寻的成本，但由于分类粗细度低，且功能层次仅类似于书签，往往让用户难以得到想要的信息。而搜索引擎的出现，使人们可以通过关键词进行主动搜索，同时组合关键词等高级搜索功能的普及使得人们可以进行更加复杂和深入的查询^[5]。例如在淘宝、京东、Amazon 等电商网站，主页的搜索功能方便用户进行全站内的商品搜索。此外，像百度搜索、谷歌搜索等因特网搜索引擎均可以帮助用户在因特网范围内过滤出有效信息^[6]，此方式解决了信息分类的缺点，但需要用户对待过滤信息的关键字十分清晰并主动发起搜索请求。因此搜索引擎无法满足没有准确搜索目标的用户群体，而推荐系统可解决此问题^[7]。

推荐系统因具备个性化推荐能力受到了广泛的关注^[8]。目前，推荐系统在方方面面都对生活产生了积极的影响，在在线电商、视频网站、社交网站、Feed 流平台等领域都得到了普及。例如在京东、亚马逊、淘宝等电商网站，通过搜集用户的历史购买和浏览数据，个性化地对商品进行推荐，提升了相关产品的购买转换率^[9]。在 Netflix、

Bilibili、Youtube 等在线视频网站的个性化广告推荐和喜好视频推荐，延长着用户的在线观看时长。字节跳动旗下的短视频产品抖音（Tik Tok），也受到了广大网民的欢迎，并在全球范围成为了热门 APP^[10]。Facebook、小红书、Twitter 等社交网站的可能相识好友的推荐功能，也展现着推荐系统的效果^[11,12]。

综上所述，推荐系统在电商推荐、视频推荐和社交推荐等各个领域百花齐放，在给用户带来极大便利性和满意度的同时，也间接的增加了用户对各系统的粘性，进而给相关公司带来了巨大的利益。因此，学术界和工业应用领域的相关公司逐渐关注到推荐系统带来的好处，开始着力于研究推荐系统，但现有的推荐系统暴露出的个性化特征提取不足、实时性低等问题依旧存在改进空间，因此如何通过提升推荐系统的个性化推荐能力和实时性，具有重要的研究意义。

1.2 国内外研究与发展现状

1.2.1 推荐算法研究和发展现状

推荐系统作为计算机科学中一个较为新兴的研究领域，直到近 30 年来才初见端倪。明尼苏达大学计算机科学系的算法研究实验室最早开始推荐算法的研究，而协同过滤（Collaborative Filtering, CF）也是在这时得到了首次提出，并基于新的概念完成了推荐系统的初步设计^[13]。虽然该系统并没有主动为用户推荐的能力，但为今后的发展作了铺垫，ACM 也将此多次作为大会的主题^[14-16]。1997 年，MovieLens 推荐系统诞生，其包含的数据集作为经典数据集直到现如今都在被广泛使用^[17]。2003 年，Amazon 公开了其算法，并应用于电商推荐^[18]，主要是采用了基于实例的协同过滤算法（Item-based Collaborative Filtering），该算法可以对用户规模极大的物品评分矩阵进行建模，通过分析用户在亚马逊购物商城不同页面之间的行为来给用户进行精准的推荐。根据其提供的数据，推荐系统给亚马逊购物商城带来的用户占比在 20%左右。2006 年，Netflix 开启了全球第一个推荐算法竞赛，竞赛的内容为，将现有的推荐算法的效果提升 10%以上，就可以获得巨额现金奖励^[19,20]。该比赛的现金奖励丰厚，吸引了全世界的科研团队，在该比赛结束后，产生了很多经典且在当时效果很好的推荐算法。例如 Y Koren, R Bell, C Volinsky 等人提出的矩阵因式分解（Matrix

Factorization) 算法, 合并了额外的隐性反馈、时间效应和置信水平信息, 在当时取得了较好的效果, 因而广泛地运用于各大领域^[21]。

随着近些年来深度学习的快速发展和成熟, 更多的研究者将神经网络相关方法应用于推荐算法。与传统的基于协同过滤的推荐算法相比, 神经网络的学习方式可包含更多的隐含信息, 可在大量的低密度信息矩阵中, 更好地表示出有意义有价值的信息, 同时, 结合目前发展较快的大数据处理框架和分布式计算技术, 克服了深度学习中计算量大的缺点。比如, Netflix 公司就采用深度神经网络 (Deep Neural Networks, DNN) 来进行在线视频推荐。将融合处理后的视频特征属性数据和获取到的用户行为数据导入到 DNN 模型中, 并将 DNN 模型的输出导入到深度神经网络的隐含层进行学习和处理, 最终实现了对用户进行 Top-N 的推荐^[22]。2006 年, 张锋、常会友等人利用反向传播算法 (Back Propagation) 成功解决了推荐系统中常见的数据稀疏性和冷启动问题, 最终提高了推荐系统的正确性和准确性^[23]。同年, 来自多伦多大学 Hinton 等人发明了基于受限玻尔兹曼机 (Restricted Boltzmann Machine, RBM) 的协同过滤算法。由于从各方面来看, 该算法取得的成果, 比之前参与 Netflix 算法竞赛的算法都要好, 使得今后 Netflix 竞赛的算法都提升了一个档次。RBM 仅仅是由两层神经网络构成, 完全不同于传统意义上的三层神经网络模型。在此之后, Hinton 对其进行改进, 提出了深度玻尔兹曼机 (Deep Boltzmann Machine, DBM) 模型, 此改进模型符合了神经网络的特征, 并且是一个深度学习模型^[24,25]。2011 年, Murat Aykanat 等人开创性地将神经网络和传统的数据挖掘分类算法融合在一起, 将 SVM 算法和 CNN 算法进行了有机的结合, 提高了数据集分类的精确性^[26]。P Covington 等人利用深度神经网络算法构造的推荐系统为 Google 旗下的 YouTube 在线视频网站带来了巨大的流量, 该算法首先对采集到的用户数据进行建模操作, 并利用优化后的排序算法进行排序, 最后通过自动选择几种策略进行视频的推荐选择^[27]。微软公司则在微软 Edge 浏览器中的资讯推荐和微软商城的应用程序推荐上采用深度学习, 也达到了不错的推荐效果^[28]。领英公司提出了一种广义线性混合模型^[29], 全局影响因子的设计和使用是该模型最大的特点, 并且单独为用户和用户的职业设置了单独的影响因子。所以特定职业的用户可以成为该推荐算法首先考虑的对象。领英公司的推荐算法相

比于传统的推荐算法，可以调节影响因子从而使得算法更偏重于特定用户的喜好或者是不同职业的用户，进而提升该算法的效果。

在音乐推荐领域，推荐算法的研究也逐渐增多。1995 年，由 MIT 媒体实验室开发了第一个音乐推荐系统 Ringo^[30]，Ringo 向用户推荐可能喜欢的音乐，预测用户对歌曲的评分，以及对歌曲的喜好程度。2012 年举办的 Million Song Dataset Challenge，则是专门针对音乐的个性化推荐比赛，面对商业领域庞大的用户群体和音乐数量数据的稀疏性，此比赛提供了一个包含 100 万个用户收听记录的大型数据集，不仅包括用户相关信息、歌手、发行时间等基本元数据，还有标签、音频内容分析等内容，一百多个团队参加比赛并提出了许多基于内容的音乐推荐解决方案^[31,32]。基于内容的音乐推荐，就是把内容上与用户原喜欢的歌曲相近的歌曲推荐给新用户。到现在，音乐推荐在国内外都取得了相当的进展，大量的音乐网站、在线电台为用户服务。国外著名音乐网站 Pandora 和 Last.fm 是音乐推荐领域的领军代表，Spotify、Rhapsody 也致力于在线音乐服务，国内也涌现出数量众多的在线音乐产品，如豆瓣电台、虾米网、腾讯音乐等。

在国外的近期研究中，Bogdanov 等人研究了基于内容的分析，通过分解用户喜好的一首歌曲的音频，从用户模型中获得音轨然后进行高级语义的描述，为用户产生感兴趣的歌曲^[33]。Hyung 等人通过对用户输入的信息（包括用户的背景信息或者对一首音乐的描述）进行文本分析，通过用户自身的数据来对用户推荐与其相符的歌曲^[34]。还有一些研究是在基于音乐标签的推荐上面，例如，Horsburgh 等人提出了一种新的混合表示，在不直接引入内容的情况下增强了稀疏标签的表示^[35]。2017 年 Oramas 等人结合文本和音频信息与用户反馈数据，使用深层网络架构解决了如何推荐先验知识稀缺的新歌手的冷启动问题^[36]。2019 年 Kowald 等人对 LastFM 数据集进行研究，发现了流行偏见会导致长尾商品和对流行商品不感兴趣的用户受到不公平待遇的问题，为后续研究开拓了方向^[37]。2020 年 Ke 等人探讨了用户音乐播放历史数据和人口统计特征，构建了一个嵌入式用户模型来表示用户的音乐偏好，模型可通过分别计算音轨的音频嵌入和不同的用户嵌入之间的相似度来确定推荐的最佳用户组^[38]。2022 年 Park 等人提出基于项目的变分自编码器（Variational Autoencoders

Encoder, VAE) 和贝叶斯个性化排序矩阵分解 (Bayesian Personalized Ranking Matrix Factorization, BPRMF) 的模型, 模型对每个流行组使用一个基于实例的 VAE 和一个额外的正则化来减轻流行度的偏差, 取得了不错的效果^[39]。

1.2.2 大数据处理引擎研究和发展现状

近年来随着数据规模的不断增大, 大数据处理引擎的研究和发展变得越来越重要。目前, 市场上有许多大数据处理引擎, 如 Apache Flink、Apache Hadoop、Apache Beam、Apache Spark 和 Apache Storm 等。随着 Google 公司 GFS、MapReduce、BigTable 三篇论文的公布, 火热全球的大数据算法的基石就此奠定^[40-42]。GFS 是一种分布式文件系统, 旨在存储和管理大量数据。它使用多个副本来确保数据可用性, 并使用各种优化技术来提高读写性能。GFS 的主要优势之一是其高可用性和高扩展性, 但它也有一些局限性, 例如无法修改文件和缺乏文件级访问控制; MapReduce 基于分而治之的方法, 由两个核心过程组成: Map 和 Reduce。Map 过程将输入数据映射到中间结果, Reduce 过程将中间结果合并为最终结果。MapReduce 的主要优点之一是能够处理大量数据并支持高并发, 但效率相对较低, 不太适合交互式查询或实时数据处理; BigTable 是一种分布式键值存储, 它具有由行和列组成的数据结构, 用于存储大量结构化数据, 支持高并发。在 BigTable 中, 每个单元格都可以存储一个可以版本化的值, 从而允许存储多个带时间戳的值。BigTable 的主要优点之一是其高可用性和性能, 但它也有一些局限性, 例如缺少事务和对复杂查询的支持。总的来说, GFS、MapReduce 和 BigTable 是对大规模数据处理的分布式系统的设计和开发产生了巨大影响的重要系统, 以上三者解决了在分布式环境中管理和处理大量数据的挑战, 并促成了许多重要应用程序和服务的开发。Doug Cutting 在结合上述三篇论文的思想, 与分布式文件系统 (Distributed File System, DFS) 相结合, 开发出了新一套分布式基础架构解决方案-Hadoop, 并被 Apache 基金会收录。

Hadoop 的应用场景非常广泛, 其强项是分布式的架构, 常见的海量数据存储和处理场景是其最擅长的, 其架构如图 1-1 所示。

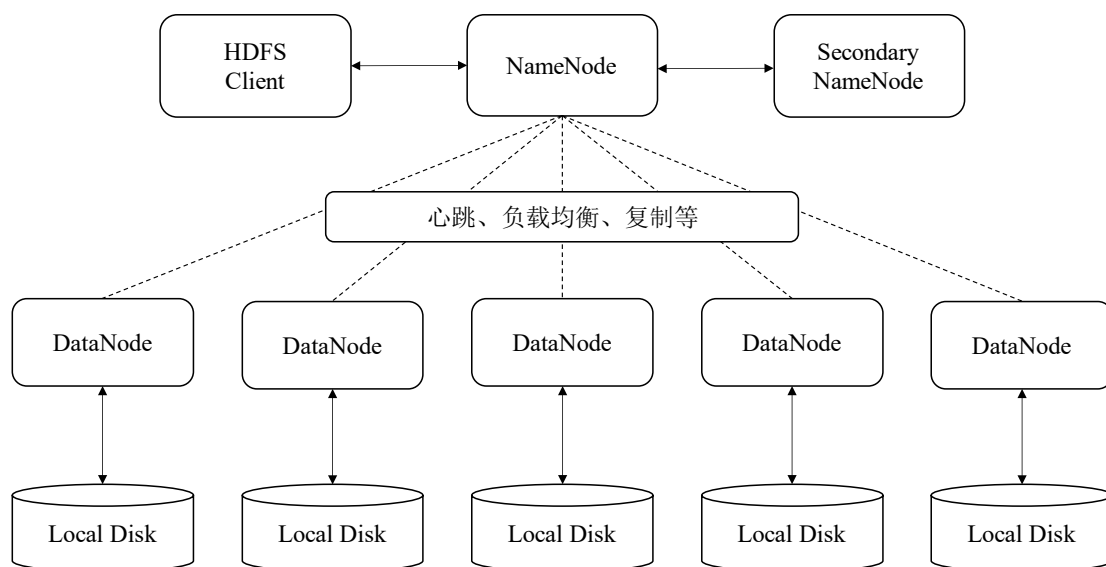


图 1-1 Hadoop 架构图

在编程实现方面，作为框架的 Hadoop 无需编程人员了解底层实现细节，其提供了大量的 API 接口可供开发人员直接使用。此外，Apache 旗下的开源项目 Mahout，依托 Hadoop 大数据平台，使用了大量 Hadoop 中现成的算法模型，例如聚类、分类和协同过滤等等，也减少着开发者的使用成本。虽然 Hadoop 具有大量的优点，但最大的缺点是基于持久存储模式的设计而导致的频繁磁盘 I/O 操作，因此愈加无法满足用户的需求^[43]。

而基于内存批处理模式的 Apache Spark 则解决了以上问题，Spark 的生态系统如图 1-2 所示。

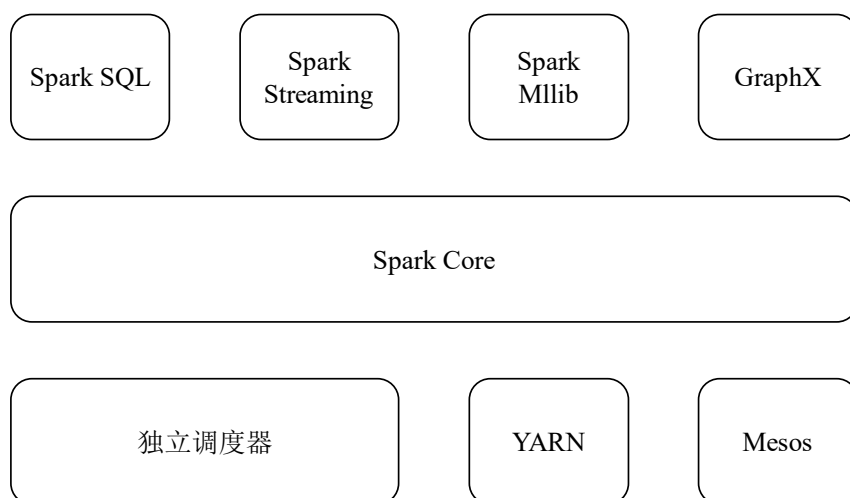


图 1-2 Spark 生态系统

Spark 提供了一组高效而灵活的 API，可以用来开发各种数据处理任务，包括流式处理、机器学习、批处理和图计算，Spark 生态中各项目的主要功能如表 1-1 所示。

表 1-1 Spark 各模块功能

模块	功能简介
Spark Core	Spark Core 是 Spark 的核心组件，实现了内存管理、任务调度、与存储系统交互等基础功能，并具有相关 API 供外界调用
Spark SQL	Spark SQL 是 Spark 中用于操作结构化数据的组件，支持 Hive、Parquet 和 JSON 等数据源，可以通过 SQL 语法来查询和修改数据
Spark Streaming	Spark Streaming 是 Spark 中用于计算流式数据的组件工具类
Spark MLlib	Spark MLlib 是关于机器学习算法的工具类，比如分类和回归等算法负责整个集群不同节点之间的信息通信以及不同资源之间的协同调度和
Spark 集群管理器	管理。Spark 对集群载体的支持很广泛，如 YARN、Apache Mesos 等都可以运行

Spark 对运行的集群管理载体要求宽泛，可以运行在大多数主流集群管理系统上，包括 Apache Hadoop、Apache Mesos 和 Kubernetes，并且支持如 Java、Python 和 R 等多种编程语言。Spark 的主要优势在于它支持在内存中进行数据处理，而不是像 Hadoop 那样将数据写入磁盘，从而能够大幅提升处理和读写数据的速度。这使得 Spark 非常适合于对实时数据进行处理、查询和转换的应用场景。此外，Spark 组件还具备一系列成熟的工具集并开放了 API 来帮助用户开发和构建各种数据处理任务。例如，Spark 提供了基于 SQL 的 DataFrame API 帮助用户处理结构化数据；提供了 MLlib 库帮助用户开发机器学习模型；还提供了 GraphX 库帮助用户开发图计算应用。总之，相比于 Hadoop，Spark 是一个非常强大的基于内存的大数据处理工具，广泛应用于各种行业，包括金融、电信、广告、医疗保健和政府部门等。

随着生产环境中越来越大规模地部署流式计算技术，现有的计算框架也逐渐地暴露出缺陷和性能问题，例如只能处理较为简单的事件而无法处理复杂事件，只能运用于较少的几种场景等等。新一代流式计算引擎 Apache Flink 的诞生解决了上述的问题，Flink 的高吞吐能力、兼容流式处理动态数据和静态数据以及批处理数据等多种应用场景的泛化能力，以及保证消息队列中的 exactly-once 语义的能力都是 Flink

的重要能力。最重要的是，在计算过程中的所有状态数据 Flink 都会定期进行快照操作并保存，从而保证了在出现问题之后的快速复原和现场恢复，提高了在异常情况下的消息处理的精确性^[44,45]。它由柏林工业大学开发，后来捐赠给了 Apache 软件基金会。Flink 旨在实时处理大量数据，通常用于实时分析、欺诈检测和事件驱动应用程序等任务，图 1-3 展示了 Apache Flink 的组成部分。

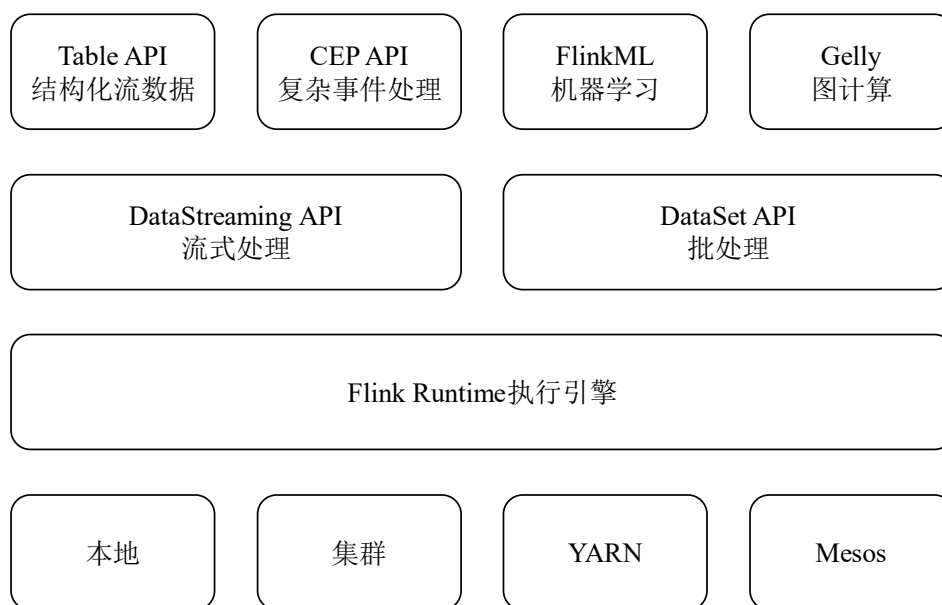


图 1-3 Flink 生态系统

它具有许多关键特性，使其非常适合流处理任务，包括：**容错性**：Flink 被设计为具有容错性，确保在发生故障时数据不丢失。**可扩展性**：Flink 能够根据应用程序的需要向上和向下扩展，使其能够处理大量数据。**低延迟**：Flink 旨在以极低的延迟处理数据，使其非常适合实时应用程序。**数据集成**：Flink 可以很容易地与各种数据源和接收器集成，使其易于在各种环境中使用。总体而言，Flink 是处理流处理任务的强大工具，被广泛应用于各行各业。

1.3 主要研究内容

论文针对当前推荐系统暴露出的个性化特征提取不足、实时性低等问题，鉴于深度神经网络在各领域表现出的优异性能和效果，结合深度学习中长短期记忆网络和卷积神经网络的特性，提出了一种可以感知用户在不同时间序列下喜好信息的个性

化音乐推荐算法（CNN-LSTM-SA）。此推荐算法利用长短期记忆网络适合处理时序特征的特点，从用户兴趣序列信息中学习用户喜好；利用卷积神经网络适合处理高维数据的特点，从非序列高维稀疏信息中学习用户和音乐之间的偏好关系；使用注意力机制增强模型对用户兴趣序列中不同信息重要性的理解效果，而提升了个性化特征的提取能力，进而完成音乐的个性化实时推荐。基于 CNN-LSTM-SA 推荐算法，以提高系统运行性能和实时性为目标，完成了音乐推荐系统的需求分析与整体架构设计，并选用 Flink 实时计算框架以及 Java EE 开发框架工具 Spring Cloud、MySQL、Kafka、Filebeat、Redis、Elasticsearch 等组件对各核心功能模块进行了详细设计与实现，系统由音乐推荐和搜索模块、数据可视化和交互模块、用户和系统管理模块组成。在系统中，基于 CNN-LSTM-SA 推荐算法构建了具备个性化实时推荐能力和离线推荐能力的音乐推荐模块；基于 B/S 架构实现了交互性良好的系统可视化和异常告警能力；使用 Flink 实时计算框架、Kafka 消息队列和 Redis 缓存提升了实时数据处理能力和实时推荐能力。

最后，通过完整的实验，测试了所采用的 CNN-LSTM-SA 推荐算法效果与个性化实时音乐推荐系统功能和性能。

论文全文共分为七个章节，每章节的内容安排如下。

第一章 绪论：简要介绍音乐推荐系统的研究背景和意义、国内外研究现状、论文的主要研究内容。

第二章 相关技术与理论：介绍本课题的主要算法，包括协同过滤算法、神经网络算法、词嵌入技术，其中着重介绍了后面章节将要使用的神经网络算法。

第三章 个性化音乐推荐算法研究：介绍基于卷积神经网络和带自注意力机制的个性化音乐推荐算法（CNN-LSTM-SA）的改进思想和设计思想，其中重点介绍算法对异构数据处理的特点和对算法效果提升的原理。然后讲解算法的运行流程和训练的过程，并对算法进行了对比测试。

第四章 个性化实时音乐推荐系统分析与设计：介绍系统的功能性需求和非功能性需求、系统架构设计、系统的功能设计和系统数据库设计，基于实时性目标，重点介绍了实时系统的解决方案，全面确定了个性化实时音乐推荐系统的设计方案。

第五章 个性化实时音乐推荐系统实现：根据算法研究、系统架构设计和功能设计，对用户和系统管理模块，音乐推荐和搜索模块，数据可视化和交互模块三大模块的实现进行详细介绍。

第六章 个性化实时音乐推荐系统测试：介绍音乐推荐系统的测试环境和数据集、评价指标，并基于系统功能、系统性能和执行效率维度对系统进行完善的测试。

第七章 总结和展望：论文内容的研究总结以及后续改进方向。

2 相关技术与理论

论文旨在使用深度学习技术设计并实现一个适用于个性化音乐推荐场景的推荐系统，其核心任务是使用长短期记忆网络和卷积神经网络从系统组件中采集异构的用户行为日志数据，并对异构数据针对性处理，从而预测用户与音乐的喜好关系。在本章中，首先介绍推荐算法领域中经典的协同过滤算法原理与特点，并以此为基础进一步介绍神经网络相关技术，其中着重介绍推荐领域中被逐步广泛使用的卷积神经网络和循环神经网络算法。

2.1 协同过滤算法

协同过滤算法是最基础的推荐算法，最早于 1992 年由 Grundy 等人提出，在近 30 年中，越来越多的研究人员参与研究协同过滤算法。其基本原理是通过历史行为和喜好预测未来的行为和喜好，即通过以往用户的各项数据记录和行为来进行预测。行为数据主要包括用户的各页面点击率、停留率、停留时长，购买信息和分享信息等序列数据。

协同过滤算法经过多年的发展已经有多种优化方案。如图 2-1 所示为协同过滤算法分类图。

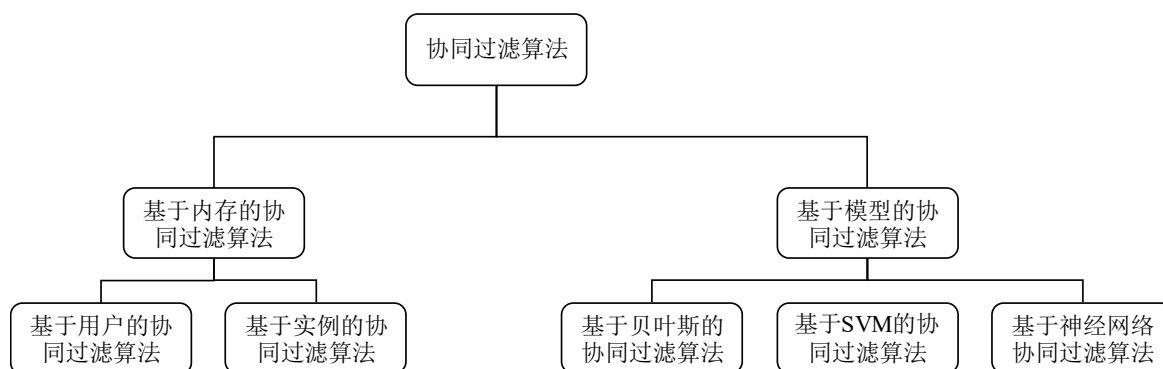


图 2-1 算法的分类

在内存之间通过相似度算法计算两个实例或对象之间的近似程度，并通过分类算法（如 K 近邻等）将相似的对象归为一类，并认同两者是相似的，是基于内存的协同过滤算法基本思想^[46]。而基于模型的协同过滤算法是借助离线计算预先通过线

性回归或者贝叶斯之类的方法训练出一个现成的预测模型，进而通过此模型进行新用户的推荐^[47]。

2.1.1 基于用户的协同过滤算法

如图 2-2 所示为基于用户的协同过滤算法（User-based Collaborative Filtering）的原理图。

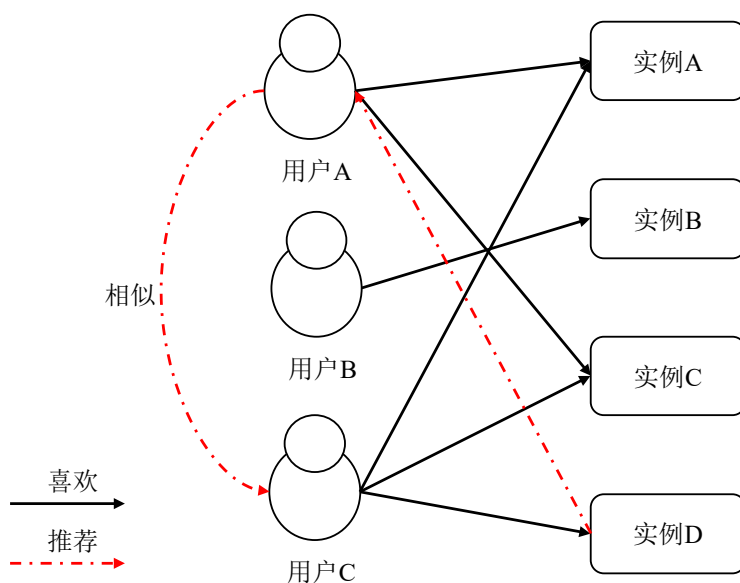


图 2-2 基于用户的协同过滤算法

基于用户的协同过滤算法建立在同类用户的喜好越接近的原则来进行推荐。其考虑的是用户之间的相似程度。过程一般分为两步，第一步用户的历史数据计算出相似度集合，第二步通过和集合内的各个对象进行对比，计算出最相似的前 N 个用户，也就是用户对实例喜好程度的排名。

2.1.2 基于实例的协同过滤算法

基于实例的协同过滤算法（Item-based Collaborative Filtering）和基于用户的协同过滤算法相似，但它们侧重点不同，前者侧重于实例之间的相似度，即物品之间的相似性。具体来说，它首先利用用户对不同物品的各项历史行为数据计算不同物品之间的相似性系数，从而得到不同物品之间的相似度关系。然后，根据用户对物品的历史反馈记录，将待推荐集合中与用户曾经喜欢的物品相似性较高的几个物品推送给当

前用户。这种方法可以有效地利用历史数据，并在给定相似度关系的情况下提出更合适的推荐物品，是一种非常有效的推荐方法，被广泛应用在电子商务、社交媒体等领域，其原理如图 2-3 所示。

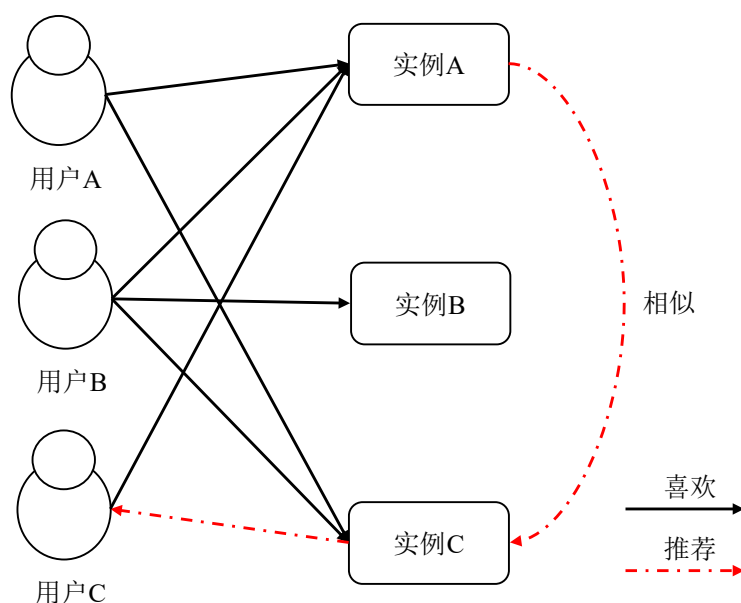


图 2-3 基于实例的协同过滤算法

基于用户的协同过滤算法，由于用户群体规模较大且用户群体的变更频率较高，每次进行推荐需要重复计算整个用户集合内的相似性系数，所以计算的时间成本会比基于实例的协同过滤算法要高许多。但是在用户群体非常庞大的情况下，可以增加用户群体分类的多样性以及推荐算法的新颖度。对于基于实例协同过滤算法，由于大部分情况下实例的集合数量相比于用户数而言不会特别的庞大，且实例的集合是较为稳定的，从而实例之间的相似度在一定的时间和空间范围内变化程度较小，因此计算出的相似度结果在一定的时间内可以复用，但其缺点在于无法为用户提供多样性的推荐和新颖的推荐。总而言之，对于用户群体较大的推荐系统而言，可以采用基于实例的协同过滤算法，而对于用户群体较小的推荐系统而言，则应该主要采用基于用户的协同过滤算法^[48]。

2.1.3 基于模型的协同过滤算法

对于大型的推荐系统而言，用户数量和实例的数量都有可能十分的庞大，对于实

时内存计算的上述两种协同过滤算法需要大量的计算硬件资源和软件资源，同时算法的计算时间和空间亦呈指数级的增长，不能满足当下互联网时代对时效性和高效存储的要求，基于模型的协同过滤算法则克服了此缺点。

基于模型的协同过滤算法最开始起源于数据挖掘技术（Data Mining），通过将大量的数据进行清洗校验，随后进行预处理（距离衡量、抽样、降维），并通过数据分析算法（预测、聚类）处理后建立模型，即可为用户进行目标推荐。不同于基于内存的协同过滤算法每次都提取用户实时的数据进行分析，基于数据集训练的模型具有一定的延时性和滞后性，并且训练出的模型效果很大程度上取决于数据集的质量。因此需要定期的对数据集进行维护，避免数据库中的脏数据和过时数据影响到推荐模型的准确性。

2.1.4 混合推荐技术

由于不同的协同过滤推荐算法有不同的适用场景和优缺点，而当今的推荐系统所要面临的推荐方式是复杂多变的，单一的推荐算法已经不能满足一个优秀推荐系统的需要。

对于一个优秀的商业推荐系统而言，几乎会有几十种不同的推荐算法协同混合工作。不同的推荐算法通过一定的混合方式能更好的适应不同的适用场景，产生比单一推荐算法更好的推荐效果，其中最主要的混合方式如表 2-1 所示。

表 2-1 混合推荐技术表

混合类型	说明
加权式混合推荐	采用多种推荐算法，并通过加权形式结合
转换型混合推荐	不同的场景采用不同的推荐算法
瀑布式混合推荐	将不同推荐算法串联，一个推荐算法的结果输出输入到另一个推荐算法中继续计算；不同推荐算法并联，针对不同的输入数据采取不同的推荐算法策略

2.2 神经网络算法

神经网络（Neural Network），是 1943 年美国神经生理学家 Warren Mcculloch 和

数学家 Walter Pitts 模仿生物神经学的机制而发明的一种数学模型^[49]。其基本原理是脑细胞的活动就像开关的通断，这些细胞通过不同的连接方式互相传递信息，从而可以进行多种逻辑运算和信息的存储。

人工神经网络基本单位神经元的结构如图 2-4 所示。

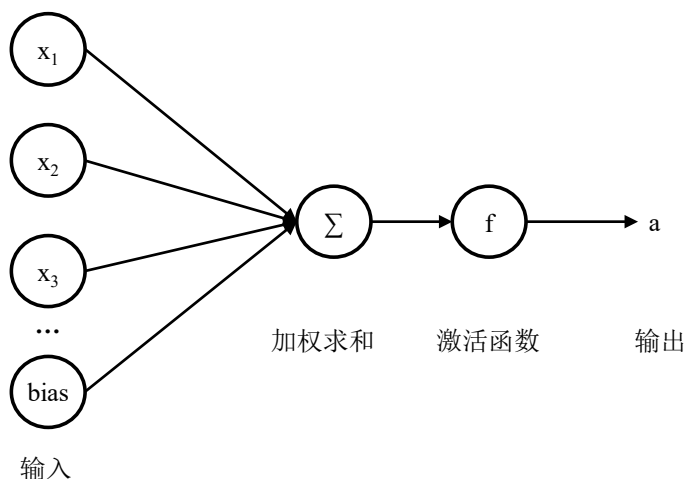


图 2-4 人工神经网络的基本单位—神经元

其中，输入层到求和层之间的有向线段代表了不同的权重，每一个输入乘以权重值再进行求和经过激活函数 f 的作用则得到了神经元的输出 a ，而 $bias$ 表示偏置。一个神经元单位有若干个数据输入 $x_1, x_2, \dots, x_n$ ，用向量 $\mathbf{x} = [x_1, x_2, \dots, x_n]$ 来表示输入，并用 z 表示输出，其公式如式(2-1)所示。

$$z = \mathbf{W}^T \mathbf{x} + b \quad (2-1)$$

其中 $\mathbf{W} = [w_1, w_2, \dots, w_d] \in \mathbb{R}^d$ 和 $b \in \mathbb{R}$ 是偏置，是神经网络待学习的参数。将 z 输入激活函数（Activation Function）之后，就得到 a ，如公式(2-2)所示。

$$a = f(z) \quad (2-2)$$

激活函数负责映射输入输出，一般取激活函数为非线性函数，例如阶跃函数、Sigmoid 函数、ReLU 函数等，从而能具有对复杂现实世界的建模能力。

深度学习已经在 CV 领域取得了不错的成果，近年来深度学习也在 NLP 方面加速发展^[50,51]。深度学习中，涉及到众多的框架和算法，表 2-2 列举了近年来主流的几种深度学习框架及其应用场景。

表 2-2 主流深度神经网络及其应用

架构	应用
递归神经网络 RNN	语音识别、手稿
卷积神经网络 CNN	图像视频处理、自然语言处理
长短期记忆网络 LSTM	文本压缩、语音识别、手势识别、图像说明

2.2.1 卷积神经网络

卷积神经网络（Convolutional Neural Networks, CNN）由 Yann LeCun 等人受到生物学上的感受野（Receptive Field）机制而提出，是神经网络的一种改进型算法，其具有局部连接，权重共享等特征，可以进行监督和非监督学习。如图 2-5 所示是典型的卷积神经网络结构。

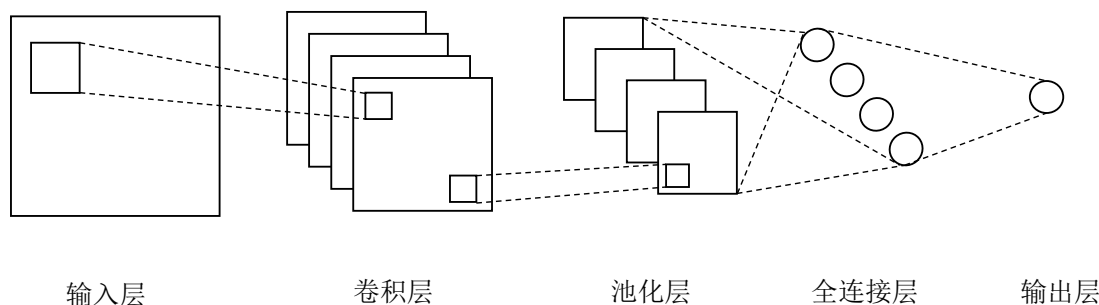


图 2-5 卷积神经网络的结构

其中池化层和卷积层是卷积神经网络的主要部分。卷积层的主要功能是提取输出信息的特征，从而对现实物理世界建模。池化层的作用是进一步抽象卷积层提取的特征，减少噪声信息，增加有效信息的比例，进而减少参数的数量，同时也可避免训练的模型过拟合。下面对上述两者做基本介绍。

卷积神经网络中的卷积操作就是按照某一要求不断更新卷积以提取最合适的特征。该过程可表示为公式(2-3)所示。

$$y_{mn} = s \left(\sum_{j=0}^{j-1} \sum_{i=0}^{i-1} x_{m+i, n+j} w_{ij} + b \right), \quad (0 \leq m < M, \quad 0 \leq n < N) \quad (2-3)$$

其中 $\mathbf{x}$ 表示输入向量，向量中的每个元素表示图像的像素值大小；图像尺寸大小为 $j \times i$ ； w 、 b 、 y 分别表示卷积核、偏置项和卷积层的输出， s 就是上述的激活函数。

池化层的作用在于丢失无用信息而保留有效的特征信息,也称为下采样层。由于已经通过卷积操作提取了特征,所以相邻区域内的特征相似,且几乎不发生变化,因此在池化层中,通常用每个邻域中像素的最大值替换邻域中的所有像素,使其变为一个像素,这样做可以增加卷积神经网络的鲁棒性,上述方法也称为最大池化。

2.2.2 循环神经网络

循环神经网络(Recurrent Neural Network, RNN)是一种特殊的神经网络结构,能够处理序列型数据,例如时间序列、语音序列等。

RNN 的核心在于,它的内部每个时间步的隐含状态都与前一个时间步的输出相关。因此,模型可以在一定程度上保留前面的信息,具有对时间序列信息的短时记忆功能。从模型结构上来看,RNN 包含了一个循环体,该循环体可以从序列中的每一个时间步接收输入数据,并生成当前时间步的隐状态和输出。RNN 内部使用一个隐藏层记录信息,并通过把上一个隐状态和当前时间步的输入进行结合,生成当前时间步的隐状态,其公式如(2-4)所示。

$$h_t = f(Ux_t + Wh_{t-1}) \quad (2-4)$$

其中, x_t 是时间步 t 的神经元的输入, h_{t-1} 是时间步 $t-1$ 的隐含层状态,是用于存储 $t-1$ 时刻信息的模块,前一时间步隐藏层的状态和当前时间步的神经元输入共同决定了隐含层状态 h_t ,函数 f 通常是一个非线性的激活函数。

因为 RNN 的上述特性,其可以较为轻易地处理时序信号,故被广泛地运用在自然语言处理的任务上,但循环神经网络不具备处理长度过大文本的能力。因此,对长序列数据的处理,通常需要采用长短时记忆网络(Long Short-Term Memory, LSTM)或门控循环单元(Gate Recurrent Unit, GRU)等技术。

LSTM 是 RNN 基础的常用变体,它能够很好的解决 RNN 的上述缺点,不仅可以学习到更多的长距离上下文信息,还可以减少 RNN 在处理高维度信息时的梯度爆炸或是梯度消失问题。LSTM 中加入了门控机制来控制信息的传递,分别为输入门、输出门、遗忘门,其中遗忘门用来防止信息过载,其公式如式(2-5)所示。

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (2-5)$$

输入门用来给隐含层输入新的信息，其公式如式(2-6)所示。

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (2-6)$$

输出门决定隐含层中的哪些信息输出到外部，其公式如式(2-7)所示。

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (2-7)$$

其中 W 、 U 、 b 是不同门控机制的权重参数， $\sigma(\cdot)$ 是激活函数，一般情况下为 Sigmoid 函数。

在 RNN 中，隐含层可以保留历史时间步的信息，并且在每一个时间步，隐含层的信息都会被更新，所以被称为短期记忆。而门控机制被运用于 LSTM 中，通过选择性地丢失和保留信息，从而确保重要信息的保留时间更长，但也不是无限地通过时间步传递下去，因此叫做长短期记忆网络。长短期记忆网络能够有效学习到更多的长距离上下文信息，但是其由于参数量大，需要较多的训练时间。

2.3 词嵌入技术

在神经网络之中，有一些文本数据是无法直接输入到神经网络中进行计算的，需要转换成向量的方式进行表示，这就是词嵌入（Word Embedding）技术。常用的方式分别为 One-hot、Word2Vec 和 FastText。

所谓 One-hot 形式，即给定一个词表，并且让每个词向量的长度都等同于词表的长度，而每个词在词表中的位置对应的词向量元素为 1，而其他元素为 0。这种词向量表示原理简单，但是缺点明显，每个单词对应的词向量的维度会非常高，并且在高维的词向量中，只有一个位置存有有效信息。因此信息密度低，任务的整体效果不佳。其次，对于同一个单词，此表示方式无法体现出单词的多义，词性和上下文信息。因为上述缺点，人们开始寻找一种可以替代 One-hot 的表示方法，并且这些表示方法可以较小维度去表示单词，从而高效地体现出单词的多义、词性和词之间的上下文信息，于是提出了词向量矩阵，而词向量矩阵需要大量的数据集来训练，会比较耗时，

因此目前大多使用预训练的词向量。Word2Vec 是预训练的词向量目前较为流行的方法，是总结人类的抽象语言并转换为向量形式表示的一种方式。Word2Vec 有两个原始的任务：分别是 Skip-gram 模型和 CBOW 模型（Continuous Bag-of-Word Model），图 2-6 展示了模型的结构。

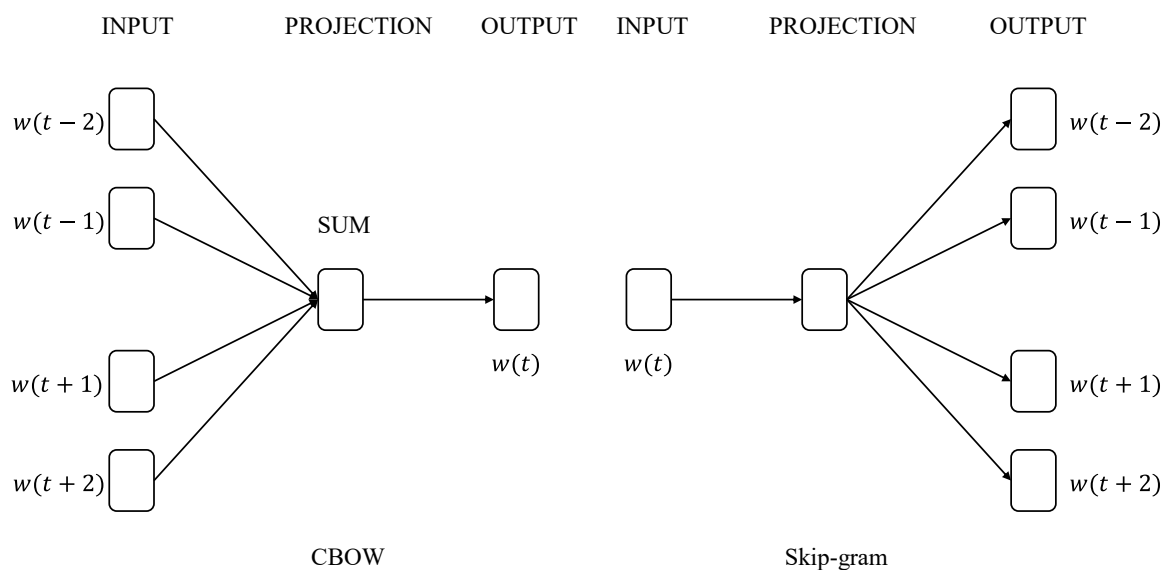


图 2-6 CBOW 模型和 Skip-gram 模型结构

其中， w 代表词向量，对于 CBOW 来说， $w(t-2)$ ， $w(t-1)$ ， $w(t+1)$ ， $w(t+2)$ 以及待训练的结果 $w(t)$ 都是单独的单词对应的向量表示，在 CBOW 模型中，隐含层进行的是一次 SUM 求和取平均的操作，训练时以 $w(t)$ 的词向量为优化目标，利用梯度下降算法训练至收敛，得到训练好的词向量矩阵。设定训练范围的大小为 a ， W 代表词向量，可以用公式(2-8)表示。

$$W_t = \frac{1}{2a} \sum_{i=1}^{2a} W_i \quad (2-8)$$

在训练过程中，公式中的 W_t 通过代价函数不断地进行优化，从而不断地更新参数。在训练完成后得到的词向量矩阵，就是 Word2Vec 方法预训练好的矩阵，可直接使用。而 Skip-gram 模型的训练过程则相反于 CBOW 模型。Word2Vec 虽然提高了文本表示能力，但是它也存在着明显的局限性。例如，基于分布式表示策略的 Word2Vec 需要基于词表构建一个包含所有单词的词向量表示的向量空间。所以，它无法识别和处理词表之外的 OOV（Out of Vocabulary）单词。另外，由于浅层神经网络不具备参

数共享能力，这使得词根相同的单词也无法被正确识别。

FastText 就是基于 Word2Vec 的改进，它充分利用单词的内部结构，在训练过程中基于单词上下文更高效构建词向量空间^[52,53]。FastText 在词向量的无监督训练过程使用的是和 Word2Vec 相同 CBOW 架构或 Skip-gram 架构，但是在此基础上，FastText 增加了 N-gram 的特征，该特征包括两种：

- (1) 词与词之间的 N-gram 特征。
- (2) 单个词内的 sub-word 特征，即字符级别的 N-gram 特征。

FastText 使用 N-gram 特征不仅可以有效表示 OOV 单词，还可以为字符组成相近的稀有单词构建词向量。因为字符级的 N-gram 特征可以根据词表中单词构造出与某个单词相似的罕见词或者不存在于词表中的 OOV 词。通过这种方式可以有效利用单词间相同词根的参数信息优化单词词向量的训练质量，也可以扩大词表构建更大范围的词向量空间。另外，如果仅以词表单词进行词向量表示，这样就完全忽视了文本中词的上下文信息。而利用 N-gram 特征可以有效关联文本中相邻的单词，使得在词向量训练过程中捕获单词的局部顺序信息，让模型训练出更高质量的词向量。

2.4 本章小结

本章主要介绍了个性化实时音乐推荐系统所涉及的主要理论和相关技术。首先对推荐算法领域经典的协同过滤算法的原理和特点进行了分析，接着介绍了近些年来广泛运用于推荐算法的深度学习神经网络方法，其中重点介绍了用于时间序列数据提取的长短期记忆网络、用于高维特征提取的卷积神经网络，最后详细介绍了特征提取使用的词嵌入技术。

3 个性化实时音乐推荐算法研究

本章介绍基于卷积神经网络和带自注意力机制的长短期记忆网络的推荐算法 CNN-LSTM-SA, 说明所提出算法涉及到的模型设计和算法计算流程, 并重点介绍算法对异构数据处理的特点和对算法效果提升的原理。最后介绍算法实验设计、算法实验过程和算法实验结果分析。

3.1 问题描述

推荐算法通过收集用户和实例之间的源关系数据, 经过数据的预处理和适配, 输入到特定的模型中进行处理, 最终在关系数据中找到用户和实例之间的相似关系, 并进行匹配的过程。目前推荐算法在很多领域已取得很大进展, 但也面临着诸多技术挑战, 例如:

(1) 多源异构数据如何高效处理和融合以获得更多个性化信息。推荐系统中包含有多种来源且结构不同的数据, 例如, 用户和实例的行为数据不仅包含了经常动态变化的评分时序数据、喜好时序数据、浏览时序数据等, 还包含了用户属性、实例属性等相对变化频率较低的数据, 如何处理和利用好推荐系统中多源异构的数据是当下的技术挑战之一。

(2) 时序信息如何充分获取。如图 3-1 所示, 某场景下用户 A 的音乐播放记录为 Music1→Music2→Music3, 而用户 B 的音乐播放记录为 Music1→Music3→Music2, 如何通过此时序信息获取到两个用户不同的喜好信息也是当下的技术挑战之一。

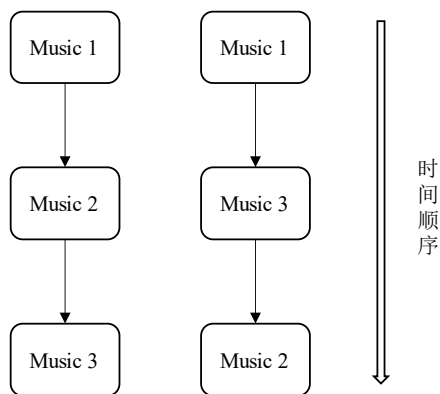


图 3-1 时间顺序的影响

针对上述问题，对现有的推荐算法和音乐推荐系统的数据结构进行了分析和研究可知，输入到音乐推荐算法中的数据可分为静态特征数据和动态特征数据。静态特征数据是指对象的静态属性，比如一个用户的性别、年龄、性别、国家等诸如此类变化频率相对较低的数据，而动态特征数据则一般用来指用户的历史行为数据，是一个时间强相关性的数据，比如用户对某音乐的交互数据（音乐播放、音乐评论、音乐收藏）等等。由于一个数据集中对象的静态特征数据和动态特征数据彼此分离，互不影响，因此可以采取并行处理的方式和不同的策略方法处理这两类特征数据，即通过对同一组数据的不同特征输入到更适合特征本身特性的算法中进行数据处理和计算，最终进行合并处理并获取结果，从而追求更高的数据处理速度和模型质量。根据上述思路，受深度学习中长短期记忆网络适合处理时序特征的特点和卷积神经网络适合处理高维数据特点的启发，对于音乐推荐系统而言，用户在系统上产生的数据可通过两种神经网络分别进行处理：

（1）长短期记忆网络处理用户行为序列数据

用户行为序列包括用户播放序列，用户点赞序列，用户浏览序列，用户收藏序列，由于这些数据是时间性强相关的，因此是一个典型的时序数据，考虑采用长短期记忆网络处理这部分数据。

（2）卷积神经网络处理非序列高维数据

非序列高维数据包括用户性别、用户年龄、用户国籍、音乐名称、音乐歌手和音乐标签等特征性较强但几乎不发生变化的数据，考虑采用卷积神经网络来处理这部分数据。由于此数据的变化频率较低，经过卷积神经网络处理后的结果可在数据未变化时进行保存，减少了整个算法的数据计算量，从而提升了算法的运行效率。

根据异构数据的特点和不同深度神经网络模型的特点对算法模型进行针对性的结合，解决了多源异构数据如何高效处理和融合的问题和时序信息获取的问题，从而提高模型的效果。

3.2 CNN-LSTM-SA 推荐算法设计

根据以上分析，论文提出了一种基于卷积神经网络和带自注意力机制的长短期

记忆网络的深度神经网络算法(CNN and LSTM with Self-Attention, CNN-LSTM-SA), 包括输入和数据预处理层, CNN 层, Bi-LSTM 层, 自注意力层和输出层, 其框架结构如图 3-2 所示。

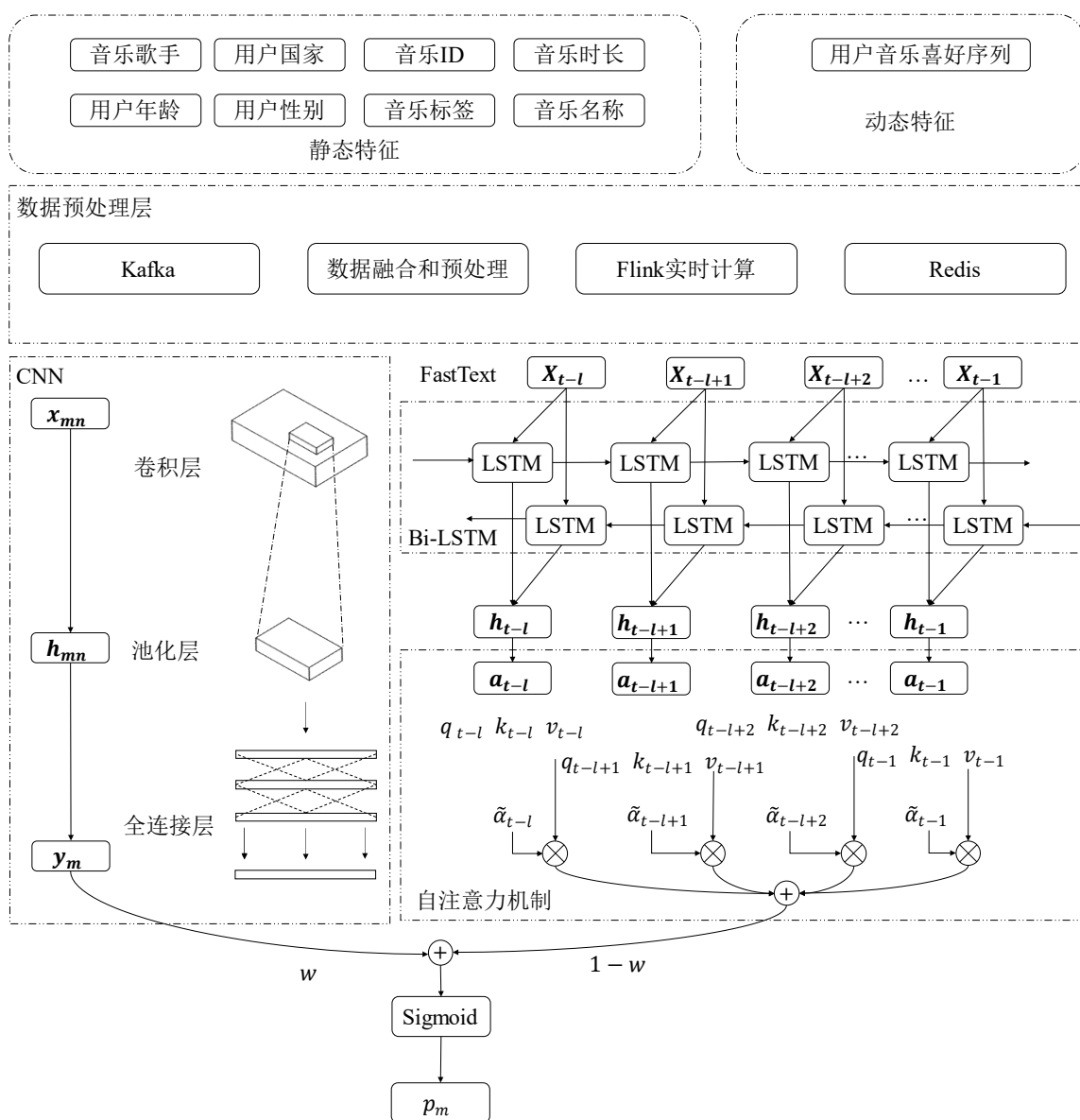


图 3-2 算法架构图

下面将对 CNN-LSTM-SA 算法的五层分别进行介绍:

(1) 输入和预处理层

推荐算法输入层分别有两种类型的数据, 第一是用户行为产生的时间强相关的序列数据, 取用户近 l 个对音乐播放的行为序列作为预测序列, 即 $\mathbf{T}_x =$

$[X_{t-l}, X_{t-l+1}, X_{t-l+2}, \dots, X_{t-1}]$, $T_X \in \mathbb{R}^{l \times m}$, 其中 m 表示每个序列的维度, 是用户近 l 次音乐播放序列中对不同音乐的喜好程度以及音乐名称、音乐歌手、音乐标签的词向量拼接。例如在 t 时刻用户播放了音乐 A , 则 $X_t = [score_A, x_A, p_A, t_A] \in \mathbb{R}^m$, 其中 $score_A$ 表示喜好分数, x_A, p_A, t_A 分别表示音乐名称、音乐歌手、音乐标签的词向量, 计算方法如式(3-1)所示。

$$score_i = \frac{n_i}{l} \quad (3-1)$$

其中, n_A 表示的是音乐 A 在序列 l 中出现的次数。

第二是用户和音乐相关的特征数据, 包括用户性别、用户年龄、用户国籍、音乐名称、音乐歌手、音乐标签和音乐时长, 这部分数据将由第二章中介绍的词嵌入 FastText 方式转换为 N 维的词向量 $x_n \in \mathbb{R}^N$, 并通过向量拼接的形式组合成矩阵 $x_{mn} \in \mathbb{R}^{M \times N}$ 作为 CNN 的输入。在使用 FastText 构建音乐静态数据中单词的词向量时, 根据 Wikipedia 提供的语料库数据预训练的向量空间获取对应单词的向量表示。词向量计算过程如公式(3-2)所示。

$$\mathcal{L} = -\frac{1}{K} \sum_{k=1}^K y_k \log(\text{Softmax}(B A x_k)) \quad (3-2)$$

其中, x_k 表示的是该文档基于词向量的文本表示, y_k 表示的是对应的特征标签, A 和 B 均为权重矩阵。

(2) Bi-LSTM 层

在第二章循环神经网络算法的介绍中提到, 原始的 RNN 模型随着深度的增加, 其在反向传播训练过程中会出现长期依赖问题。而 LSTM 通过引入了门控机制用于选择数据特征的保留和遗忘, 有效结合了长期记忆和短期记忆信息, 能够更好的学习长距离特征依赖关系, 避免了过拟合的情况。由于用户未来的兴趣很有可能对当下时刻产生影响, 因此使用了双向长短期记忆网络进行模型的训练。Bi-LSTM 层输入的每一个序列 $X_t \in \mathbb{R}^m$, 会得到两个分别代表正序列的隐含状态 $\vec{h}_t \in \mathbb{R}^z$ 和逆序列的隐含状态 $\overleftarrow{h}_t \in \mathbb{R}^z$, 拼接后得到 $h_t \in \mathbb{R}^{2 \times z}$, 其中 z 代表隐含层的特征向量维度, 其计算过程如公式(3-3)至(3-5)所示。

$$\vec{h}_t = LSTM_{forward}(X_t, \vec{h}_{t-1}, \vec{c}_{t-1}) \quad (3-3)$$

$$\overleftarrow{h}_t = LSTM_{backward}(X_t, \overleftarrow{h}_{t-1}, \overleftarrow{c}_{t-1}) \quad (3-4)$$

$$h_t = \vec{h}_t \oplus \overleftarrow{h}_t \quad (3-5)$$

(3) 自注意力层

为了更加充分地挖掘用户行为序列中的上下文信息，模型在 Bi-LSTM 层获得中间隐含状态的基础上，引入了自注意力机制来增强对上下文中的重点信息的学习捕获能力，从而为用户行为序列中的数据分配不同的权重。通过自注意力模块，不仅可以增强算法的效果，还能抑制噪声数据对预测结果的影响，其计算过程如公式(3-6)至(3-8)所示。

$$q_t = W^Q a_t \quad (3-6)$$

$$k_t = W^K a_t \quad (3-7)$$

$$v_t = W^V a_t \quad (3-8)$$

其中，自注意力模块会将隐含层状态 $h_t = a_t \in \mathbb{R}^{2 \times z}$ 使用三个权重矩阵 W^Q , W^K , $W^V \in \mathbb{R}^{1 \times (2 \times z)}$ 通过矩阵变换分别得到 q_t , k_t , $v_t \in \mathbb{R}$.

利用 q_t 分别与 k_{t-l} , k_{t-l+1} , ..., k_{t-1} 运算得到 $\alpha_{t,t-l}$, $\alpha_{t,t-l+1}$, ..., $\alpha_{t,t-1}$, 其计算过程如公式(3-9)至(3-11)所示。

$$\alpha_{t,i} = \frac{q_t k_i}{\sqrt{d}}, i \in [t-l, t-1] \quad (3-9)$$

$$\widetilde{\alpha}_{t,l} = \text{Softmax}(\alpha_{t,i}), i \in [t-l, t-1] \quad (3-10)$$

$$b_t = \sum_{i=t-l}^{t-1} \widetilde{\alpha}_{t,l} v_i \quad (3-11)$$

最终得到输出 $B_T = [b_{t-l}, b_{t-l+1}, \dots, b_{t-1}] \in \mathbb{R}^{1 \times t}$, 并通过公式(3-12)所示，将注意力层的输出映射得到一个 0 到 1 之间的预测评分 p_{LSTM} , 其中权重矩阵 $w^T \in \mathbb{R}^{t \times 1}$, b 是偏置。

$$p_{LSTM} = \text{Sigmoid}(B_T w^T + b) \quad (3-12)$$

(4) CNN 层

根据上一部分输入层所述, $x_{mn} \in \mathbb{R}^{M \times N}$ 作为卷积神经网络的输入, 其计算流程如公式(3-13)所示。

$$y_{mn} = f \left(\sum_{j=0}^{j-1} \sum_{i=0}^{i-1} x_{m+i,n+j} w_{ij} + b \right), (0 \leq m < M, 0 \leq n < N) \quad (3-13)$$

其中 x 表示输入向量,即 $x_{mn} \in \mathbb{R}^{M \times N}$; $j \times i$ 表示卷积核的大小; w 、 b 、 y 分别表示卷积核、偏置项和某一层卷积层的输出, f 是 Relu 激活函数。

再将卷积层的输出输入到池化层,本算法采用的是最大池化的方式,其过程类似于卷积操作。再者,将池化层的输出输入到全连接层,以充分地交融学习到的特征,并最终通过权重矩阵和Sigmoid激活函数将 CNN 层的输出映射得到一个 0 到 1 之间的预测评分 p_{CNN} 。

(5) 输出层

根据上文分析, CNN 层和 LSTM-SA 层输出的都是预测的评分值,因此需要将两个神经网络预测评分值进行合并处理,得到最终的预测评分值。将简单地采用归一化的方式进行加权平均处理,其公式如式(3-14)所示。

$$p_{result} = w * p_{CNN} + (1 - w)p_{LSTM} \quad (3-14)$$

其中 w 是权重值, p_{result} 为最终结果, p_{CNN} 为卷积神经网络模型预测的结果, p_{LSTM} 为循环神经网络模型预测的结果。其中权重值可作为超参数,在模型训练的时候多次测试即可得到较优值。

3.3 CNN-LSTM-SA 算法训练过程

模型的超参数设置如表 3-1 所示。

表 3-1 神经网络超参数设置

参数名	参数含义	参数取值
cnn_depth	CNN 卷积核个数	50
cnn_stride	CNN 卷积核每次移动的步长	3
cnn_padding	CNN 卷积边缘填充的数值	1
lstm_hidden_size	LSTM 网络隐藏层特征向量维度	128
lstm_num_layers	LSTM 网络隐藏层的层数	2
batch_size	一次训练的序列数量	256

续表

learning_rate	学习率	0.0001
epoch	默认训练次数	100

在 CNN-LSTM-SA 模型训练的过程中,一次训练会取出数据集中 n 个用户周期为 l 的样本序列作为输入(例如 A 用户按照时间先后的近 100 次评分数据作为动态数据),模型将学习数据的特征,并输出对当前音乐的评分预测值。

论文通过 Adam 优化算法^[54]来更新模型参数,使用均方误差损失函数作为评分预测的损失函数,其思想是计算值和实际值的差值的平方的平均值,公式如式(3-15)所示。

$$loss(Y, f(X)) = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2 \quad (3-15)$$

神经网络算法的主要训练流程如图 3-3 所示,其中初始化模型参数是对数据集 中的数据 进行预处理和适配的过程。

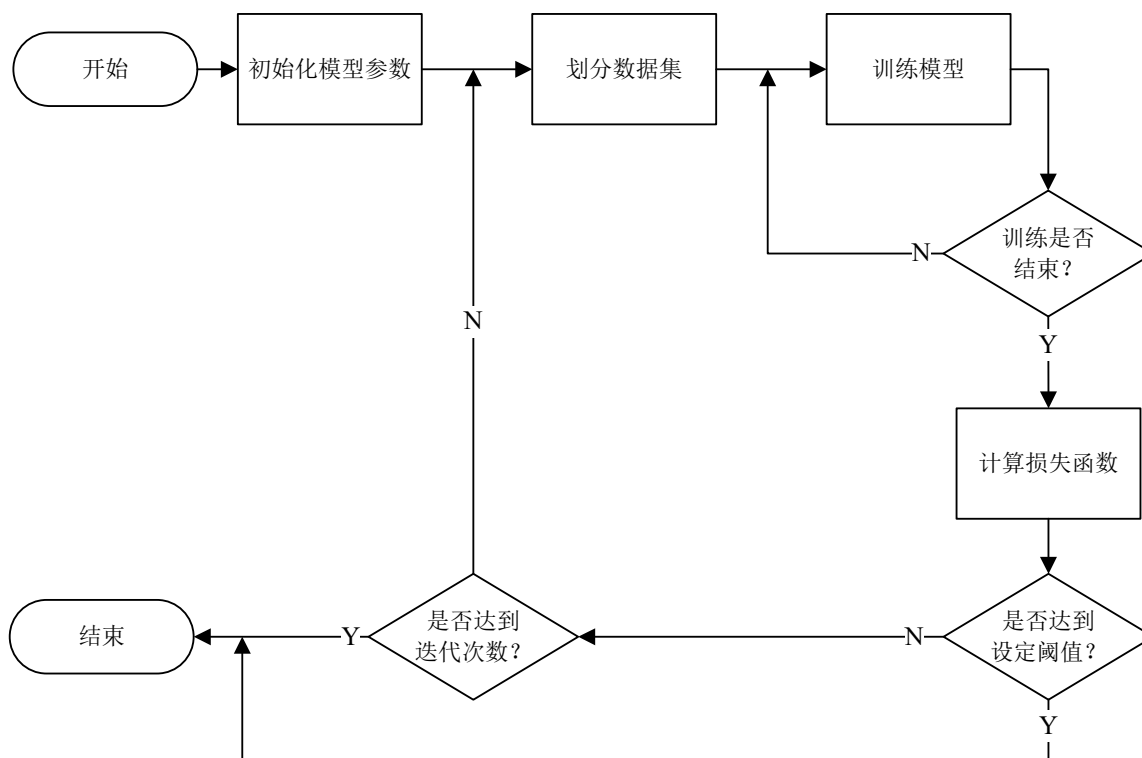


图 3-3 神经网络训练流程图

经过上述两个神经网络的分别处理和融合，最终达到了针对性地数据处理效果，从而在理论上增强了推荐算法的执行效果，CNN-LSTM-SA 模型的伪代码如算法 3-1 所示。

算法 3-1 CNN-LSTM-SA 算法

Algorithm 3-1 CNN-LSTM-SA 算法

Input: 静态和动态数据 $params$

Output: 评分预测序列 P_{list}

```
1.  Data prepare
2.  Initialize  $params$ 
3.  Initialize  $P_{cnn}$ 
4.  Initialize  $P_{lstm-sa}$ 
5.  Initialize  $P_{list}$ 
6.  for all  $param$  in  $params$  do:
7.    If  $param$  = static data then:
8.       $P_{cnn} = f(CNN)$ 
9.    end if
10.   If  $param$  = dynamic data then:
11.      $P_{lstm-sa} = f(LSTM)$ 
12.   end if
13.    $P_{cnn-lstm-sa} = wP_{cnn} + (1 - w)P_{lstm-sa}$ 
14.    $P_{list} = list.add(P_{cnn-lstm-sa})$ 
15. end for
16. return  $P_{list}$ 
```

3.4 CNN-LSTM-SA 算法实验及分析

本小节主要研究的是论文提出的 CNN-LSTM-SA 算法的超参数设定以及和其他算法的效果对比。实验主要采用对比的研究方法，比较超参数设定对 CNN-LSTM-SA 算法的影响以及 CNN-LSTM-SA 算法和其他算法在准确度上的差异，为了科学地评估上述实验的效果，采用了同一的数据集和评价指标来进行算法的评估。

3.4.1 实验数据集

系统采用 Last.fm 提供的数据集, Last.fm 是一个著名的音乐网站, 拥有庞大的用户群体和丰富的音乐数据。Last.fm 提供了开放 API, 以供研究者能够进行实验分析。论文选取 Last.fm 发布的数据集 Last.fm 1K, 其收集了共 992 个用户约四年时间内的全部收听历史记录。Last.fm 1K 主要包含用户信息表和用户历史交互记录表, 其中用户信息表包括用户 ID、性别、年龄、国家和注册时间, 用户历史交互记录表包括用户 ID, 收听时间戳、音乐歌手 ID、音乐歌手名称、音乐 ID 和音乐名称, 共有 19150868 条记录。同时使用提供的 API 对数据集中的音乐数据进行补充, 构建出包含音乐名称、音乐歌手、音乐时长、音乐标签等数据的音乐 csv 文件。对以上数据的处理采用的是 Python 的文件处理和科学运算包, 分块读取三个文件到内存中进行处理, 并进行以下操作:

- (1) 对相同用户的收听历史记录按照时间先后顺序进行排序
- (2) 对用户根据用户 ID 进行排序
- (3) 对音乐按照其演唱歌手进行分类

经过以上操作将原始文件数据转换成合适的格式存储到 MySQL、Redis 和 ES 中。

3.4.2 实验评价指标

评估测试结果将通过以下几个指标来进行:

(1) 评分预测误差

很多场景下, 评分预测都是推荐系统的重要应用, 比如商品的评分预测、音乐的评分预测、图书的评分预测等。此场景的核心在于根据当前用户过去一段时间内的历史行为记录, 来预测未来用户遇到某一个新的对象时, 会给出怎样的评分。

对于此时推荐系统的效果度量, 一般是采用用户预测评分和用户真实评分的均方根误差 (Root Mean Squared Error, RMSE) 来进行。其计算公式如式(3-16)所示。

$$RMSE = \frac{\sqrt{\sum_{u, i \in T} (r_{ui} - \hat{r}_{ui})^2}}{|T|} \quad (3-16)$$

其中, T 为测试集, 用户为 u , 实例为 i , r_{ui} 是测试集中用户 u 对实例 i 的评分真实

值，而 $\hat{r}_{ui}$ 为推荐系统预测的用户 u 对实例 i 的估计值。

通过上述公式易知，当均方根误差越小，代表推荐系统预测的分数越接近用户的真实分数；反之，当均方根误差越大，代表推荐系统的效果越差。在实际生产环境中，一般会进行归一化处理。

(2) 召回率、精确率、F1 值

在另外一种场景下，比如 Top-N 的推荐系统就无法采用上文所介绍的 RMSE 评价指标来评估推荐算法的优良程度。因为此种场景在提供推荐服务时，需要将内部推荐系统计算并排序后的 Top-N 列表展示给用户，而不是单一的评分预测。这种场景常用的评测主要通过精确率（Precision）和召回率（Recall）以及其两者结合的 F1 值进行综合考量，而论文的系统采取的推荐方式即是 Top-N 列表推荐，故论文将采用这三个值进行算法性能评估。

至于召回率和精确率的计算，二分类混淆矩阵是召回率和精确率的常见方法，用户产生点击、喜欢等正向行为时为正类，而用户忽略页面、不喜欢等负向行为时为负类，如表 3-2 所示为一个二分类混淆矩阵。

表 3-2 二分类混淆矩阵

	真实为负	真实为正
预测为负	TN (True Negative)	FN (False Negative)
预测为正	FP (False Positive)	TP (True Positive)

通过二分类混淆矩阵，召回率可表示如式(3-17)所示。

$$Recall = \frac{TP}{TP + FP} \quad (3-17)$$

精确率可表示如式(3-18)所示。

$$Precision = \frac{TP}{TP + FN} \quad (3-18)$$

在评价推荐系统效果时，召回率和精确率是常用的两个指标。召回率代表着系统推荐出的物品中，用户实际上喜欢的物品所占的比例，是衡量推荐系统覆盖程度的重要指标。精确率则代表着推荐系统预测为用户喜欢的物品中，用户实际上喜欢的物品所占的比例，是衡量推荐系统准确性的重要指标。

在实际应用中,召回率和精确率之间存在一个折中的关系,召回率越高,精确率就越低,反之亦然。因此,在评价推荐系统效果时,需要考虑到这两者的平衡,选择一个综合的评价指标来评估系统的效果。

为了结合上述的两个指标,工业界提出了一个新的标准,称为 $F1$ 值, $F1$ 值将召回率和精确率进行相关系数的合并,其计算公式如式(3-19)所示。

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \quad (3-19)$$

由公式可知,召回率和精确率的综合值越大,则 $F1$ 的值越大,从而推荐系统的效果越好。

针对本章中介绍的音乐推荐系统,最常见的场景是推荐系统给出 Top-N 符合用户的推荐列表,因此 Top-N 命中率指标常用来评价此场景推荐系统算法的效果。

Top-N 命中率的定义为:推荐系统针对 M 个用户,每个用户单独给出 N 个实例数量的排序推荐列表,对于此推荐列表,用户产生点击行为的个数占总推荐列表个数的比例即为 Top-N 命中率。其计算方式如式(3-20)和式(3-21)所示。

$$f(m) = \begin{cases} 1, & v_m \in vlist_m \\ 0, & v_m \notin vlist_m \end{cases} \quad (3-20)$$

$$TopNrate = \frac{\sum_{m=1}^M f(m)}{M} \quad (3-21)$$

其中, M 是用户的总数, v_m 表示第 m 个用户的点击记录, $vlist_m$ 是针对第 m 个用户,推荐系统计算而出的推荐列表。Top-N 命中率可以较好的评估在 Top-N 场景下推荐系统的推荐效果。

评估推荐系统的效果是至关重要的,以此来了解系统的表现如何,是否达到预期。然而,评估推荐系统的效果有很多难点,因为它不像图像分类、回归预测等其他任务那样有明确的正确性评价指标。推荐系统的效果评价涉及到多元因素,因此需要综合考虑多种评价指标来得出最终的评估结果。

3.4.3 实验过程与结果分析

系统的推荐模块采用了基于卷积神经网络和循环神经网络的推荐算法,因此需要对其中的超参数进行测试和选取,包括卷积网络层数和融合系数,以求达到最优的

推荐效果。

(1) 超参数对比实验

卷积网络层数：本组实验展示了卷积神经网络中不同的卷积层数对模型效果的影响。实验测试了从集合 $\{1, 2, 3, 4\}$ 中选取不同的卷积网络层数对模型效果的影响，实验的评价指标采用 RMSE 来评估，实验结果如图 3-4 所示。

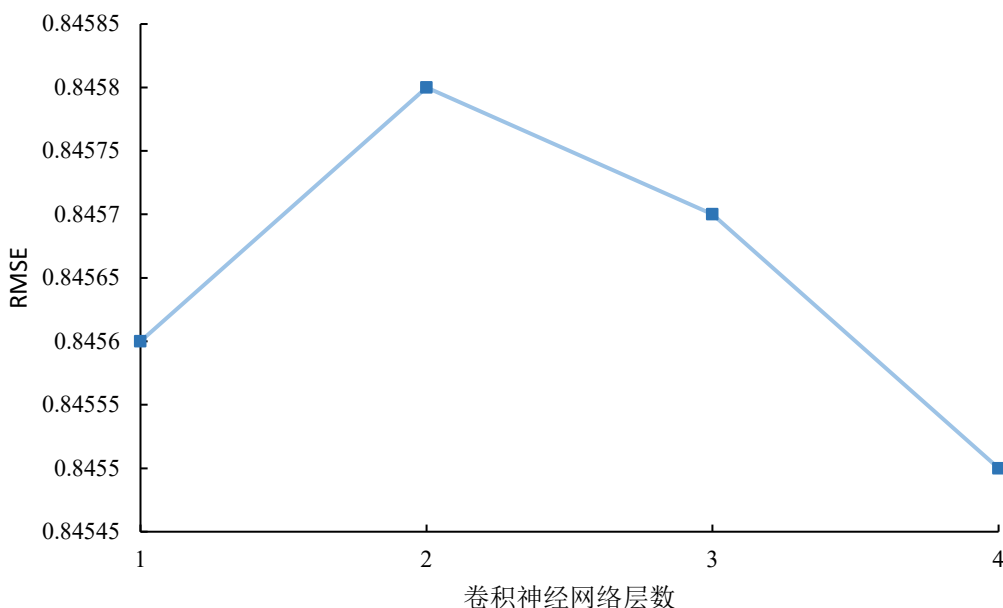


图 3-4 RMSE 和卷积神经网络层数的关系

从图中可以看出，增加卷积网络的层数有利于提高模型的表现，且在卷积层数为两层和三层的情况下，模型的表现都比单层的表现要好，而当继续增加卷积层数时，模型发生过拟合现象，不具备较好的泛化能力，因此论文将选取三层卷积神经网络进行训练。

融合系数：在前面的讨论中，采用 CNN 和 LSTM-SA 两个神经网络的预测结果的融合完成算法推荐，其中融合公式如式(3-22)所示。

$$fusion = \lambda R_{cnn} + (1 - \lambda) R_{rnn} \quad (3-22)$$

其中， λ 为融合系数，其取值范围为 $[0, 1]$ ， R_{cnn} 和 R_{rnn} 两个参数分别为卷积神经网络和长短期记忆网络的输出。

图 3-5 至图 3-7 中，展示了 CNN-LSTM-SA 算法在基于 Last.fm 数据集、Pytorch

计算框架下以及融合公式(3-22)的情形下,融合系数 λ 与精确率、召回率以及综合指标 $F1$ 之间的关系。

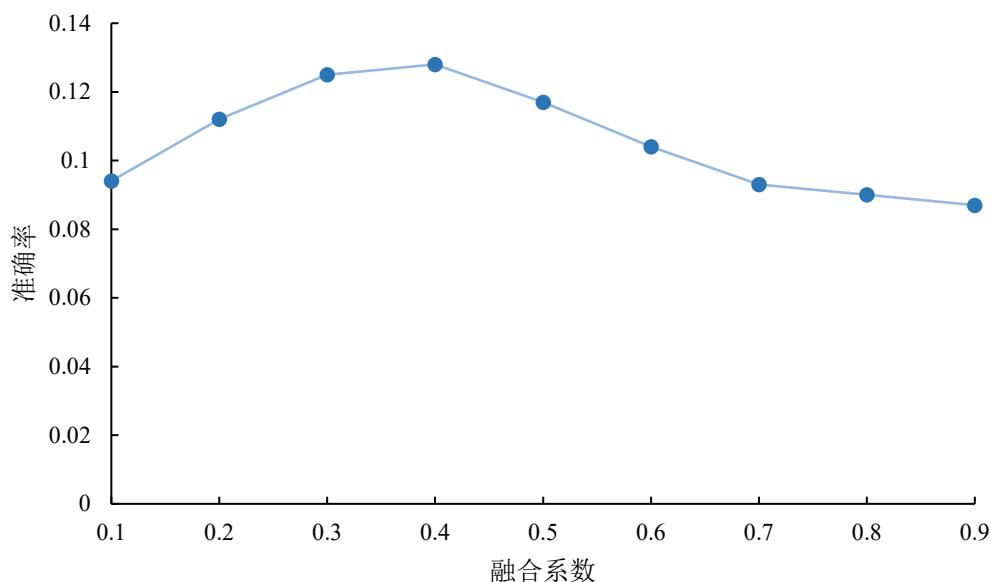


图 3-5 融合系数和精确率的关系

从图 3-5 和图 3-6 中可以看到,随着融合系数的逐渐增大, Precision 的值在随着增大后显著降低,实验中最合适的融合系数为 0.4 左右。随着融合系数的逐渐增大, Recall 的值在随着增大后显著降低。实验中最合适的融合系数为 0.5 左右。

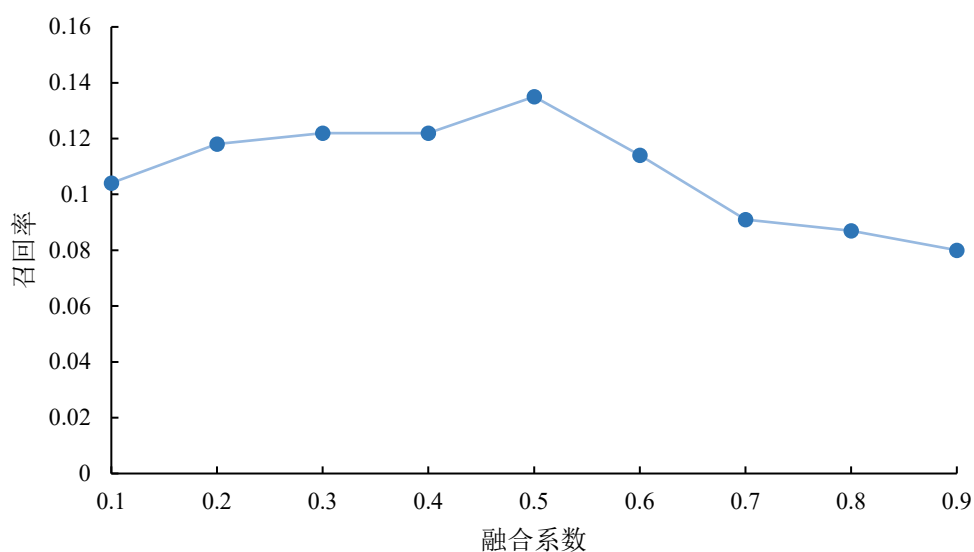


图 3-6 融合系数和召回率的关系

从图 3-7 中可以看到，随着融合系数的逐渐增大，F1 的值在随着增大后显著降低。实验中最合适的融合系数为 0.4 左右。

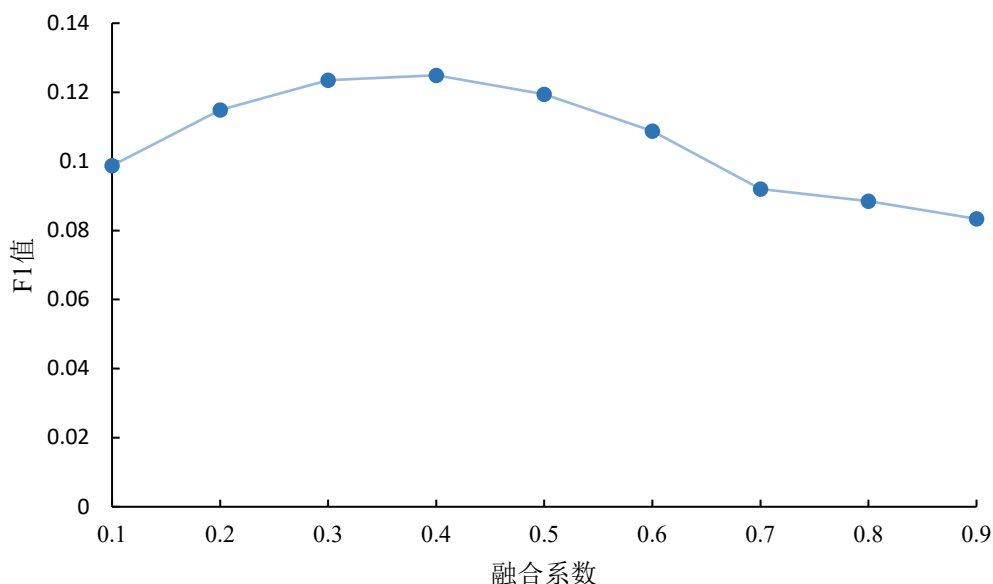


图 3-7 融合系数和综合指标 F1 的关系

(2) 模型对比实验

论文对照了推荐算法的经典模型以及推荐系统领域较为优秀的神经网络模型和本模型的实现效果，各模型介绍如下：

Collaborative Filtering: 推荐系统的鼻祖算法-协同过滤算法，施乐公司的帕拉奥图研究中心提出一种基于协同过滤的推荐算法，并将其用于垃圾邮件过滤^[55]。

Neural-CF: 首次利用深度神经网络实现协同过滤算法的一种模型，其网络结构主要分为两部分：通用神经网络矩阵分解 GMF 部分和多层感知机部分^[56]。

Deep-CF: 提出深度学习网络来预测用户-物品交互矩阵，该模型认为利用评分矩阵的评分预测的推荐算法主要分为用户和物品特征的学习以及评分预测函数的学习两部分。于是该模型利用两个神经网络分别学习用户和物品的特征信息以及评分预测函数。具体实现为在 Neural-CF 的基础上将 GMF 部分替换为 DMF 框架^[57]。

Semi-Personalized Music Recommendation (SPMR): 一种半个性化的推荐策略的音乐推荐算法，基于深度神经网络结构和来自不同信息源的用户聚类，提升了冷启动用户在未来音乐喜好推荐的有效性^[58]。

CNN-LSTM-SA: 本模型。

CNN-LSTM: 本模型去除自注意力机制模块的模型。

为保证公平性,实验参数主要参考原论文或开源代码中提出的模型中默认的最佳参数,客观地对比各模型的实验效果。将一次输入训练数量 batch size 设为 256,学习率设为 0.0001。对各模型进行预训练时,发现 90 个回合模型基本能达到稳定收敛状态,故设定上述各模型均训练 90 个回合并取平均值。

上述算法在前 N 个推荐结果 (Top-N) 上的命中率对比结果如表 3-3 所示。

表 3-3 Top-N 命中率对比

模型	N=5	N=20	N=50
CF	22.3%	40.9%	67.3%
Neural-CF	33.1%	59.1%	83.4%
Deep-CF	35.2%	62.3%	84.7%
SPMR	35.1%	60.4%	84.5%
CNN-LSTM	34.8%	59.3%	85.8%
CNN-LSTM-SA	35.3%	60.2%	86.3%

另外,上述算法在 Last.fm 数据集上预测单个实例的准确度和训练时长对比如表 3-4 所示。

表 3-4 单实例精确率对比

模型	精确率(%)
CF	11.3
Neural-CF	13.9
Deep-CF	14.5
SPMR	16.7
CNN-LSTM	16.9
CNN-LSTM-SA	17.6

由对比实验在 Last.fm 数据集上的 Top-N 命中率可见,系统提出的基于卷积神经网络和带自注意力机制的长短期记忆网络结合的推荐算法在 Top-50 命中率的算法效果上优于其他三个算法;CNN-LSTM-SA 方法的效果略优于 CNN-LSTM,说明采用

注意力机制提升了算法的效果。由单实例准确度可见，系统提出的 CNN-LSTM-SA 算法在单实例准确度的预测效果上会优于其他三个算法。

基于神经网络的协同过滤算法 Neural-CF 和其改进版本 Deep-CF 在本次算法测试中的结果均优于传统的协同过滤算法 CF，符合现有的研究成果。并且基于神经网络的协同过滤算法的结果显著优于传统的协同过滤算法，充分说明了神经网络在特征提取方面的优越性，也为本模型的理论解释提供了实验数据的支撑。论文新提出的基于卷积神经网络和长短期记忆网络的推荐算法能更好地提取到音乐相关信息的特征以及学习到用户历史记录中的时间序列变化信息，在 Top 50 的命中率上优于其他对比算法。

综合上述实验结果，论文提出的 CNN-LSTM-SA 模型可有效地提高推荐算法的准确度，证明了该模型的优势和有效性。

3.5 本章小结

本章对基于卷积神经网络和带自注意力机制的长短期记忆网络的混合算法 CNN-LSTM-SA 进行了详细地说明，介绍了模型的架构设计和算法的计算过程，其中着重介绍了 CNN-LSTM-SA 算法对异构数据处理的特点和数据处理方式对算法效果的提升，解决了个性化特征提取不足的问题。最后说明了实验设计，并使用公开数据集 Last.fm 对算法的效果进行了实验评估，实验结果显示，模型的 Top 50 命中率在所选数据集上为 86.3%，可以有效地进行个性化音乐推荐。

4 个性化实时音乐推荐系统分析与设计

本章将结合软件工程的知识，基于上一章节提出的 CNN-LSTM-SA 推荐算法，对个性化实时音乐推荐系统的系统需求进行深入的分析。然后基于架构设计的理念和实时性的目标，详细地说明推荐系统的整体模块、架构设计、功能设计和数据库设计，目的在于实现一个实时性、高效、可扩展、低成本且可行性较高的个性化实时音乐推荐系统。

4.1 音乐推荐系统需求分析

在本节中，根据当前分布式系统的发展现状，从系统的功能性需求与非功能性需求两个方面进行了分析。

4.1.1 系统功能性需求分析

功能性需求指的是用户直接感知到的需求，是一个系统的基础。音乐推荐系统的目的是帮助使用者迅速找到自己感兴趣的音乐，面向的是广大的音乐收听群体。因此音乐推荐系统首要具备的是音乐推荐功能，以满足个性化主动推荐的需求，其次要有音乐查询功能，便于用户在明确自己需求的时候主动查询音乐相关信息并且查看相关的相似音乐。另外，音乐推荐系统应采集用户与音乐之间的交互行为数据，以得知用户与音乐之间的相似关系，以及音乐与音乐之间的相似关系，这些交互行为数据作为推荐算法的输入数据，是推荐算法工作的重要基础。最后，音乐推荐系统应提供个人信息管理功能和系统管理功能，以便对个人用户数据和系统进行管理。综上所述，音乐推荐系统的具体的需求如下：

（1）音乐推荐和音乐查询

音乐推荐是系统的核心功能，可根据用户与系统中音乐的历史交互信息，使用推荐算法为用户提供感兴趣的推荐音乐列表。系统的音乐推荐功能分为三大部分，分别是实时推荐，离线推荐和热门推荐。实时推荐功能，对实效性的要求比较高，需要在短时间内更新音乐推荐列表，故采用周期性训练的 CNN-LSTM-SA 算法模型来实现，是基于当前用户的行为数据，模型以 72 小时为训练周期更新。离线推荐功能给用户

提供更加准确的推荐，同样采用 CNN-LSTM-SA 算法，并以当前用户触发离线推荐功能时刻为时间标准，以用户历史固定周期的所有行为数据进行计算，生成音乐推荐列表，算法的计算数据量大，时间复杂度相对较高，故以离线功能提供。热门推荐是统计所有音乐的播放数据，并进行简单的数据分析而排序的功能。音乐查询分为分类查询和模糊正则化查询，由于每个音乐都有对应的标签、歌手和名称，可通过这些数据对音乐在数据库 SQL 层面进行简单的分类查询。另外，系统通过 Elasticsearch 搜索引擎的检索能力以实现正则化的模糊查询能力。

（2）数据可视化和交互

数据可视化和交互是用户和音乐相关信息的管理模块，注册用户可看到音乐数据可视化的图表，同时可以再次对音乐进行播放、评论和收藏等操作。此外，注册用户可对个人信息修改，包括注册、登录、账户密码修改和绑定信息修改等功能。

（3）用户和系统管理

用户和系统管理包括用户、音乐管理功能和系统数据监控和异常处理、告警功能，本功能主要给系统管理员进行使用。用户、音乐管理功能包括用户的增删改查操作，音乐信息管理包括新增音乐、查看音乐、修改音乐相关数据等。系统数据监控和异常处理、告警功能是系统平稳运行的重要保证，包括系统运行的数据监控和用户的数据监控。此外，管理员可以收到异常的告警并进行异常处理和服务降级。

通过上述的功能需求分析，对于系统而言，主要有三类使用者，分别是管理员，注册用户和访客。游客作为用户的子集，其功能性需求也是注册用户的子集，游客的用例图如图 4-1 所示。

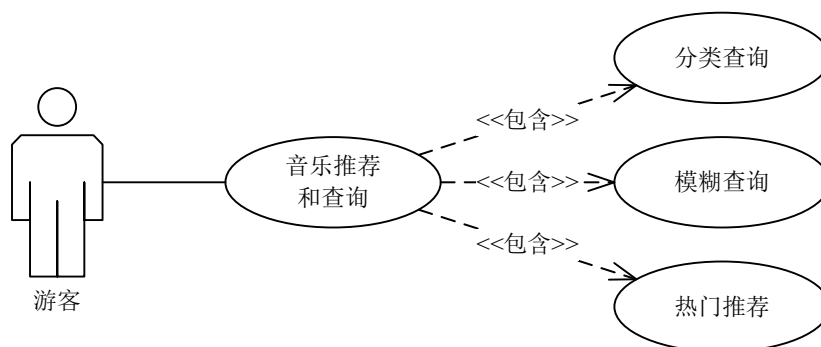


图 4-1 游客用例图

管理员可以使用上述功能需求的所有功能，相当于是超级注册用户，管理员的用例图如图 4-2 所示。

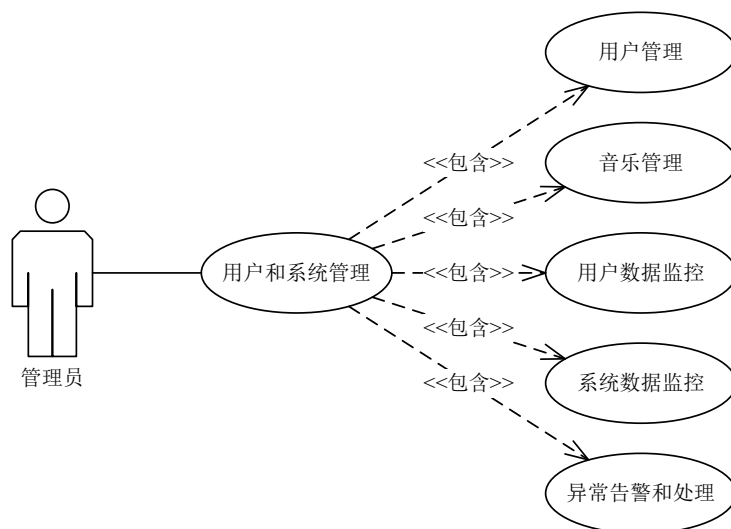


图 4-2 管理员用例图

最后是注册用户，注册用户的用例图如图 4-3 所示。

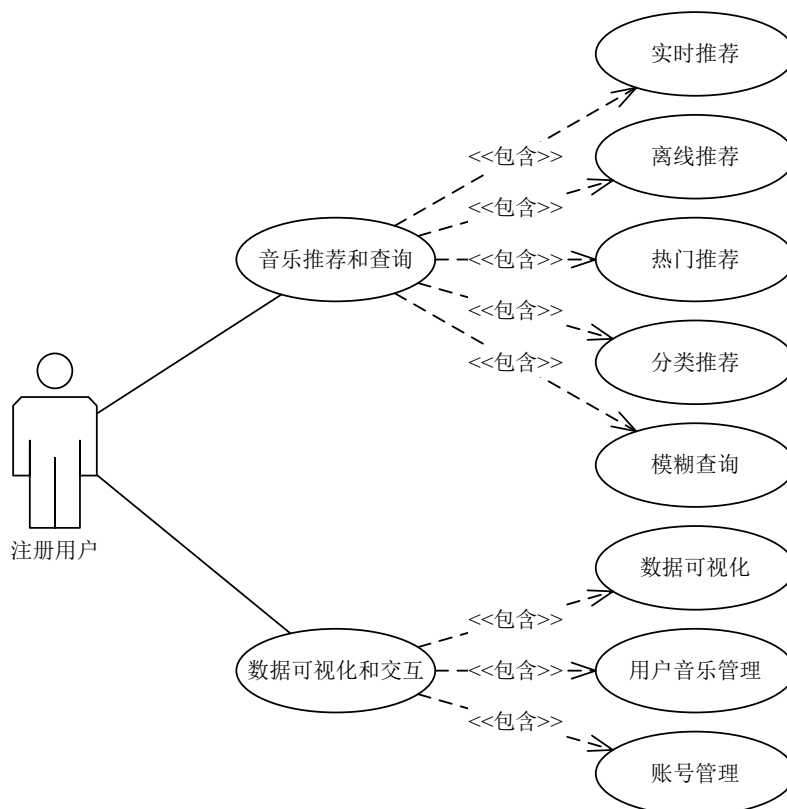


图 4-3 注册用户用例图

4.1.2 系统非功能性需求分析

系统的非功能性需求主要包括实时性、安全性、易用性、通用性、可靠性和可扩展性、成熟性和先进性。

（1）实时性

当前大多推荐系统使用离线推荐算法，因此实时性是系统首要追求的目标，也是本系统的特点。

（2）安全性

安全性是一个系统要考虑的非功能性需求。因此在构建系统时，考虑到对系统中的重要数据，如密码、邮箱等进行加盐加密，这样在发生数据库泄露时，也能最大程度保证数据无法被逆向破解。同时，考虑到用户的多设备登录，系统以 JWT（JSON Web Token）实现了用户多地、多设备登录的安全性。

（3）易用性和通用性

由于系统面向的人群较广，且系统的架构设计和算法实现较为复杂。为避免对用户产生迷惑性，系统在多处屏蔽了实现细节，并做到了风格统一，能够直观的展示各项数据和推荐结果，减少了用户的学习成本，基本做到新用户上手可用。系统的通用性主要体现在以下两个方面：第一是操作系统通用性和多设备通用性，当今越来越多用户使用手持设备进行浏览，系统通过 VUE 等前端技术实现了一套代码，多端适应的能力；第二是时间通用性，推荐系统可以通过用户历史行为的不断变化和数据库的不断完善，做到推荐的一致性和统一性。

（4）可靠性和扩展性

通过合理的服务编排、微服务技术和主从备份机制，实现集群内部服务之间的高可用和监控告警能力。采用服务治理、负载均衡、服务的熔断和降级技术实现在高负载情况下对用户的负面影响降至最低，进而保证用户体验。此外，对于系统功能的划分，采取了模块化的方式，方便后续对系统进行扩展。

（5）成熟性和先进性

在设计和制定本推荐系统所使用的技术栈时，充分调研了国内外先进的软件技术，并可做到实时更新和替换。

4.2 音乐推荐系统架构设计

系统架构设计是从宏观上进行设计和制定软件数据流和运行情况的一种方式。在实际的生产环境中，往往是考虑了各项优缺点和影响因素后衡量的结果，包括系统功能实现、组件组成方式、开发成本等等。此外，例如系统的功能和系统的技术栈选择、商业组件和开源组件的选择、数据存储方案的选择等等也是需要考虑的因素。

基于以上原因，本节选择常用的软件工程组件，进行有机的结合，期望较好地实现实时性系统的需求和功能。

音乐推荐系统的架构设计如图 4-4 所示，其中大体可分为三层，分别是数据层、策略层和应用层。



图 4-4 音乐推荐系统架构设计

其中，数据层负责加载最原始的数据集和用户历史行为数据，如音乐交互数据、用户行为数据等，这部分的数据会存储在如 MySQL、Redis、ES 等数据库中。其次，对于用户使用中实时产生的行为记录日志数据，会通过 Filebeat 日志收集器的方式定时收集并通过 Kafka 消息队列的形式进行削峰填谷后存储到 Redis 当中供实时推荐使用。策略层作为整个推荐系统的核心，起到承上启下的作用，对下利用处理和准备好的数据，进行模型的训练和运行，对上负责提供合适的接口，以便应用层进行调用。

（1）数据存储和加载解析模块

作为整个系统的数据来源，系统主要采用三种数据库。其中，业务数据库采用最为广泛使用的 MySQL 数据库，主要负责业务各项数据和配置数据的存储。对于搜索功能，采用业界常用的 Elasticsearch 解决方案，主要负责音乐数据的存储，以便用户在使用搜索功能时，能够使用最小的计算量并快速响应用户的查询需求。对于实时数据的存储，采用 Redis 键值对数据库，由于 Redis 是内存数据库，可以较快地完成数据的吞吐和计算，以便满足实时推荐算法极高的短时数据吞吐需求，提高整体系统的处理效率和并行处理能力。例如，Redis 存储了实时推荐算法中最关键的音乐和用户的交互行为数据。

（2）离线推荐、实时推荐和热门推荐模块

离线推荐、实时推荐和热门推荐是系统的三种推荐方式。实时推荐和离线功能均采用上一章节介绍的基于卷积神经网络和带自注意力机制的长短期记忆网络的推荐算法（CNN-LSTM-SA），实时推荐对实效性的要求比较高，需要在短时间内更新音乐推荐列表，故采用周期性训练的 CNN-LSTM-SA 算法模型来实现，是基于当前用户的行为数据，模型以 72 小时为训练周期更新。离线推荐功能给用户提供更加准确的推荐，同样采用 CNN-LSTM-SA 算法，并以当前用户触发离线推荐功能时刻为时间标准，以用户历史固定周期的所有行为数据进行计算，生成音乐推荐列表，算法的计算数据量大，时间复杂度相对较高，故以离线功能提供。热门推荐是统计所有音乐的播放数据，并进行简单的数据分析而排序的功能。

（3）系统业务模块

主要通过 Spring Cloud 和 Java EE 构建微服务后台框架和业务逻辑代码编写，例如用户的登录注册退出功能、音乐的推荐和搜索功能、数据的可视化和交互功能等等，部署在 Apache Tomcat 服务器上。

（4）人机交互界面

主要负责用户使用系统的各项功能，为了方便一套代码多端使用，选用 VUE 进行实现，并部署在 NGINX 服务器上。

4.3 音乐推荐系统功能设计

根据上述的需求分析和架构设计，基于 CNN-LSTM-SA 算法的音乐推荐系统主要由三大模块组成，分别是用户和系统管理模块、音乐推荐和搜索模块、数据可视化和交互模块，具体功能模块的结构如图 4-6 所示。

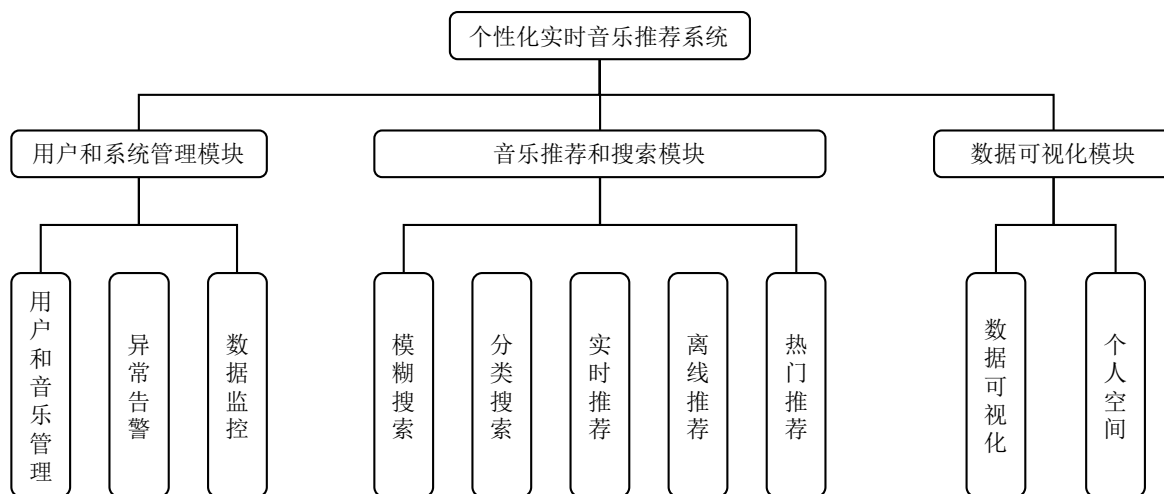


图 4-6 系统功能结构图

4.3.1 用户和系统管理模块设计

用户和系统管理模块，是全系统权限最高的模块，主要用于给管理员进行系统的监控和告警查看以及用户和音乐的信息管理所使用，因此进入本模块和在本模块的所有操作都需要进行权限校验以防止误操作或恶意攻击。

用户和音乐的信息管理功能，按照本章 4.1 节的需求分析，主要是对用户和音乐信息进行增删改查，其原理和 SQL 语句都较为简单，在此不作赘述。

数据监控功能和异常告警功能，系统使用了中间件 Prometheus 来进行实现，具体的流程如下：

- （1）Prometheus server 定期从活跃的目标主机上拉取监控指标数据，目标主机的监控数据可通过 Pushgateway 的方式采集并上报到 Prometheus server 中。
- （2）Prometheus server 把采集到的监控指标数据保存到数据库。
- （3）Prometheus 采集的监控指标数据按时间序列存储，通过配置报警规则，把触发的报警发送到 Alert manager。

(4) Alert manager 存储告警信息并配置报警接收方，发送报警。

(5) Grafana 接入 Prometheus 数据源，以方便监控数据可视化展示。

Prometheus 实现数据可视化和告警的工作流程如图 4-7 所示。

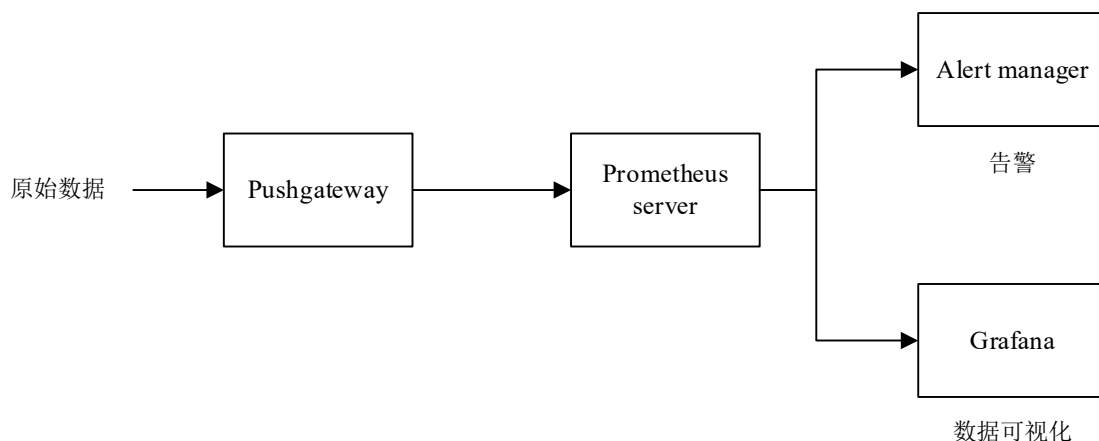


图 4-7 数据可视化和告警流程

4.3.2 音乐推荐和搜索模块设计

推荐功能主要是从海量的音乐数据中找出与用户兴趣爱好相似的音乐，系统为了更深层次和多角度地了解用户的兴趣爱好，使用了推荐模型来进行音乐推荐，推荐功能分为实时推荐、离线推荐和热门推荐。

实时推荐算法由于对时效性要求较高，所以采用了周期性训练的 CNN-LSTM-SA 模型配合 Flink、Redis 缓存来完成，且系统通过 Filebeat 日志采集的方式和 Kafka 消息队列的形式解耦了用户对音乐播放的后续操作，以减少用户播放音乐时的系统延迟感。此外，由于用户播放的行为频率较高而用户触发实时推荐的频率相对较低，且只有每次触发实时推荐才会激活系统再次运行 CNN-LSTM-SA 算法，因此将用户的音乐播放数据保存于 Redis 之中以便于快速查询和使用，由于其推荐结果为 Top-N 音乐列表，因此将其推荐结果存储于 Redis 的 List 结构中以节省内存，其 key 为“RECOMMENDLIST|用户 ID”字符串，而 value 即为推荐音乐列表 ID 的列表。考虑到异常情况，Redis 中存储推荐音乐列表的过期时间为 72*3 小时，因此用户每次触发实时推荐只需要从 Redis 中获取音乐列表详细即可。这样实时推荐算法的运行和用户的触发操作也相互解耦，符合了系统设计中“高内聚，低耦合”的原则。这种方式

可以减少用户在触发实时推荐时的计算复杂度，从而加速实时推荐的计算效率，

离线推荐算法功能开放给对精准度要求较高的用户，每一次触发离线推荐功能都需要重新训练 CNN-LSTM-SA 算法并使用最新的数据运行算法，需要的时间较长，故称作离线推荐功能，在线推荐和实时推荐工作流程如图 4-8 所示。

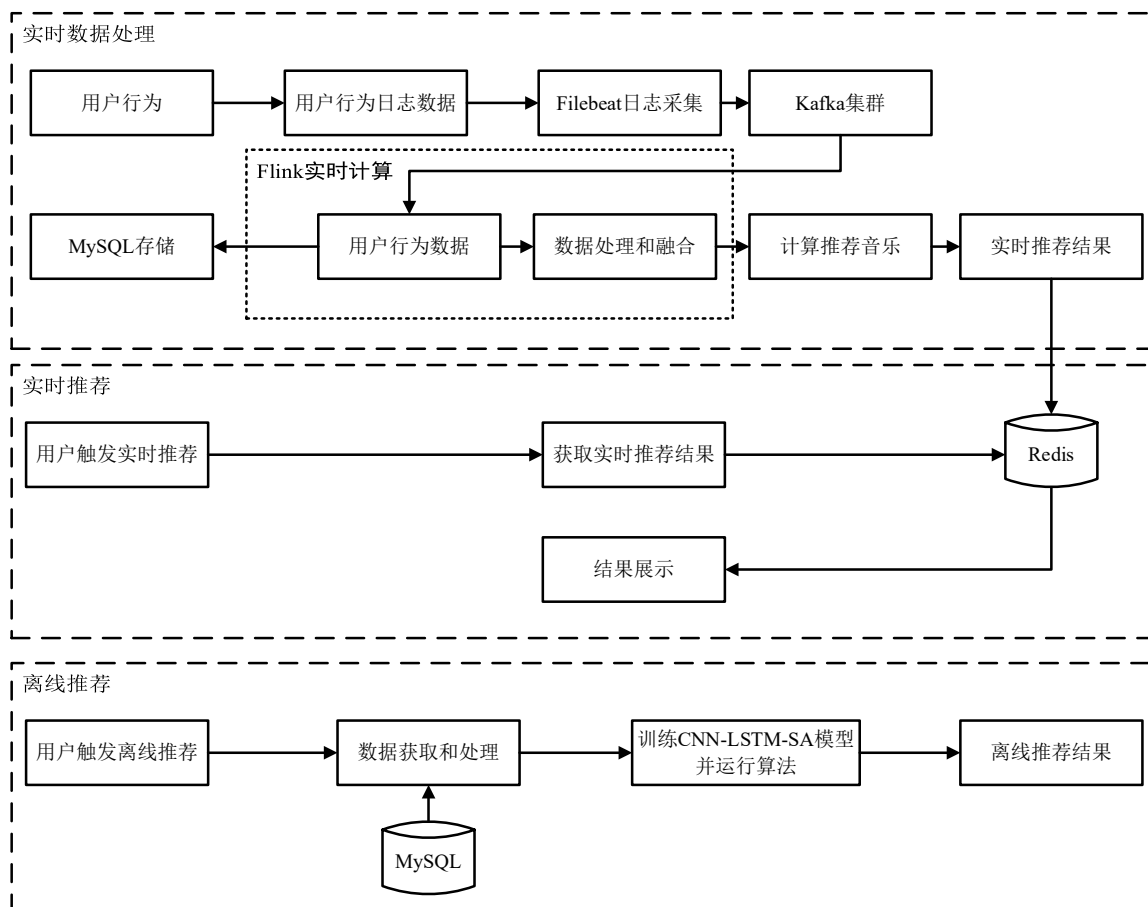


图 4-8 在线和离线推荐工作流程

对于用户行为信息，当用户无显式播放某个音乐时，会隐式收集用户在某个音乐页面的停留时间和点击次数并存储于日志中，并通过日志收集器 Filebeat 进行定期采集和上报工作。在这种情况下，系统会通过 logService 类中的 logFilter()方法对隐式的用户行为日志进行过滤和降噪。最主要是过滤单次点击和停留时长小于 3s 的用户行为，以及短期内频繁点击并且单次停留时长小于 3s 的行为，以免对用户的兴趣建模产生噪声影响。另外，由于用户的行为会随着时间的推移而变化，Filebeat 日志收集器会定期删除 6 个月及以前的所有用户日志，避免日志冗余带来的系统磁盘容量

不足的情况和过时日志对用户数据分析的影响。

当用户对收藏某个音乐时，系统首先会检查登录态的状态，若未登录则会跳转登录页面引导用户进行登录操作并重定向回收藏页面，让用户进行收藏操作。用户的收藏信息会存储到 MySQL 表中，用户的收藏流程图如图 4-9 所示。

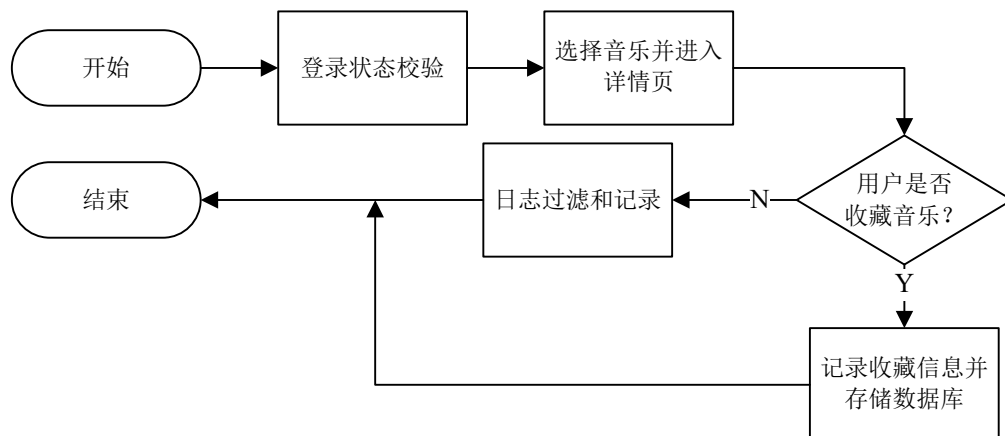


图 4-9 用户评分流程图

热门推荐功能是根据计算所有音乐的播放次数数据实现的，按照数量从高到低为用户推荐，这是数据库层面即可完成计算和过滤的功能。

音乐信息查询模块主要是音乐搜索功能，搜索功能是在首页为用户提供的，包括标签搜索和正则化模糊搜索。标签搜索的原理是根据音乐的分类信息，对数据库中的所有音乐进行分类和过滤，在数据库的 SQL 层面实现简单的标签分类搜索功能。正则化模糊搜索通过 Elasticsearch 全文搜索引擎实现，由于 Elasticsearch 提供了 REST 风格的 HTTP 接口，因此对于 Elasticsearch 的存储，删除，修改和查询都可直接调用 HTTP 接口进行。对于音乐信息索引的存储，其流程如图 4-10 所示。

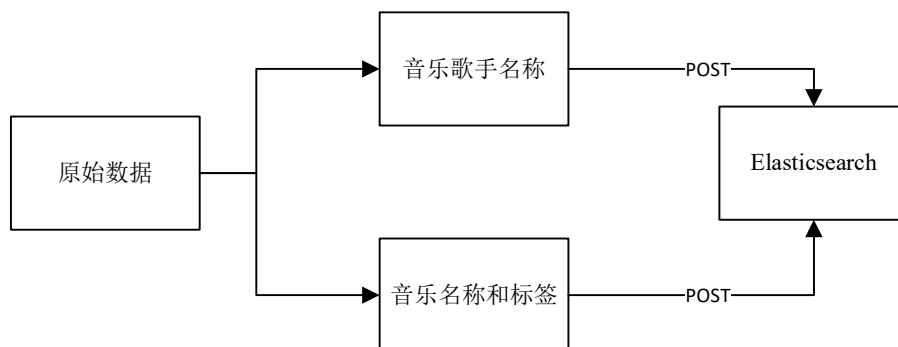


图 4-10 音乐信息存储到 ElasticSearch 流程

模糊搜索的具体实现是通过 Elasticsearch 提供的 match query 特性实现模糊搜索，该方式会对匹配文本进行分词然后匹配分词后的每个词项。例如“Taylor Swift”，Elasticsearch 会生成多个倒排索引。最基础的 key 即“Taylor”、“Swift”、“T”、“S”等等，通过这些 key 即可完成模糊搜索功能。在系统中采用 MySQL 作为主要的内容存储数据库，但在模糊搜索时，为了提高用户的检索的速度和模糊搜索的多样性，论文采用了业内常用的 Elasticsearch 来实现对音乐内容的模糊查询。因此系统需要把 MySQL 数据库中的音乐相关数据要同步到 Elasticsearch 中，这就需要保证 Elasticsearch 和 MySQL 中的数据一致性，否则将会影响到系统的正常运行。系统主要采用 Canal 技术并使用增量同步方式来解决 Elasticsearch 和 MySQL 的数据一致性问题。Canal 把自己伪装成数据库的从机，通过向数据库的主机发送备份请求，当数据库的主机收到请求后，开始推送数据库的日志信息给 Canal，Canal 通过解析日志信息里的内容，将内容再同步到 Elastic Serarch 中，其主要流程如图 4-11 所示。

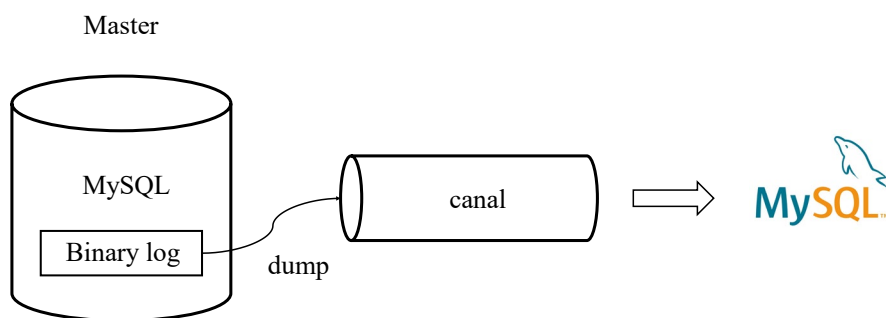


图 4-11 Canal 解决数据一致性流程

由于系统使用了 Elasticsearch 层面的缓存机制加速热点数据的查询和搜索速度，故系统在服务的层面和中间件的层面不再设计缓存以额外增加维护的成本和保持缓存一致性的成本。

4.3.3 数据可视化和交互模块设计

数据可视化和交互模块包括数据可视化、账号信息管理、音乐信息管理三大功能。数据可视化是用户行为记录的可视化展现，例如，用户的历史播放记录可视化、用户的音乐收藏记录可视化、用户的喜好变化可视化等等，其基本原理即通过 SQL 层面的数据处理来获取到图表所需要的数据，其中，在 DataController 层面会对请求

发起方的权限进行校验, 检验是否是当前用户发起对自身数据的访问, 以避免信息的泄露。在 CollectDao 层面会对用户将要执行的 SQL 进行校验, 并使用了 MyBatis-Plus 的工具以防 SQL 注入造成数据库数据异常。

对于点赞、收藏数据的查看, 其 SQL 可设计为 `select * from music_likes where user_id='current_user_id'`, 其中 `current_user_id` 表示当前用户的用户账号。用户获取点赞、收藏的时序图, 如图 4-12 所示。

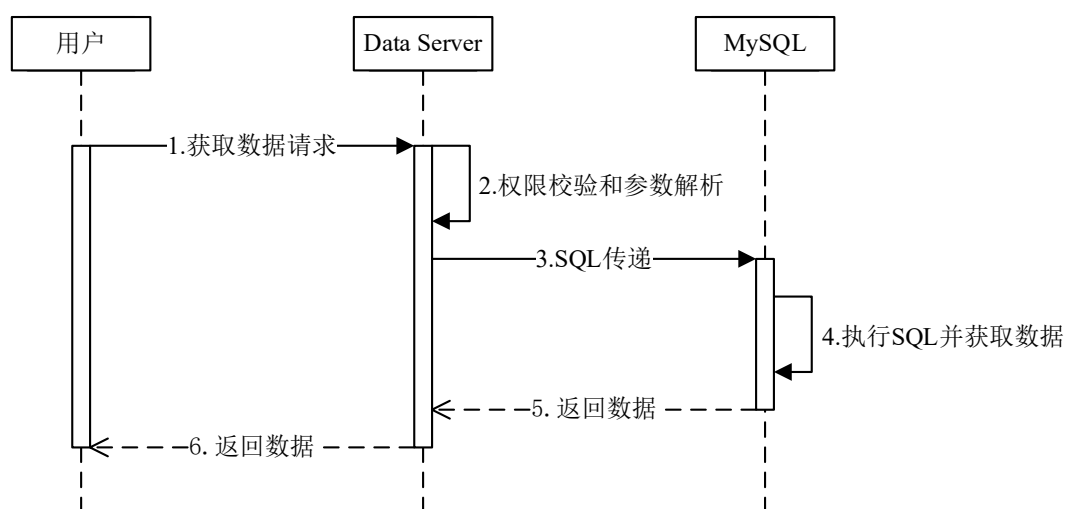


图 4-12 用户获取收藏音乐时序图

账号信息管理是对使用系统用户的个人信息数据的管理模块, 包括用户登录注册, 密码找回和首次登录后避免用户冷启动的兴趣选择等基本功能。对于用户注册而言, 首先要在前端界面保证用户输入的信息符合长度规则和格式规则, 这可使用前端的正则化校验进行。其次, 对于后端接口收到的数据请求, 也需要再次对相关数据进行正则化校验和参数解析, 以保证系统数据出错。此外, 对于账号的唯一性校验, 由 MySQL 层面的 UNIQUE 索引进行保证。用户登录和密码找回等功能的基本流程类似, 区别在于不同字段的校验有所不同, 但原理比较简单, 故在此不作赘述。

由于当前模块使用到了 Redis 内存缓存来提升用户的使用体验, 如果利用不当, 会造成缓存穿透并击垮数据库从而造成更加严重的后果。缓存穿透是指在用户读取数据的过程中缓存无效, 这样当用户的访问请求会直接绕过缓存而访问数据库, 当用户访问数量超过系统承受范围内时, 会导致系统奔溃从而引发一系列的问题, 缓存穿

透现象如图 4-13 所示。

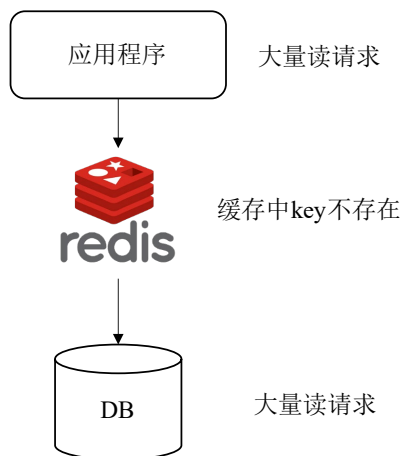


图 4-13 缓存穿透

因为缓存穿透的巨大危害，本系统在设计之初就考虑了如何避免缓存穿透问题，其流程如下：由于用户在访问缓存数据的时候都是通过数据关键字 **key** 进行访问的，因此可以通过判断关键字是否在缓存中来判断用户接下来的访问流程，这样就可以快速的返回数据信息。但是一个缓存可以容纳的数据量能到达千万级，当通过循环遍历的方式去判断关键字时，会浪费大量的时间，因此在集合中遍历的方法在时间和空间上的复杂度较高，也就违背了当初设计缓存的最终目标。为了提高访问效率，在海量数据中找关键字，需要对判断方法进行优化，因此本系统采用了业界常用的布隆过滤器（Bloom Filter）来解决这个问题。当系统采用布隆过滤器算法来对关键字进行处理时，当前端访问数据时，首先经过布隆过滤器，可以通过布隆过滤器先判断关键字是否存在，来避免用户直接访问数据库和遍历 Redis 缓存，从而解决了缓存穿透的问题。

4.4 音乐推荐系统数据库设计

数据库设计是整个系统重要的一个部分，在系统中主要使用到三种数据库。分别是 MySQL、Redis 和 Elasticsearch。

其中 Redis 是内存键值对数据库，主要作为缓存数据库，用于存储用户的热点数据，提高查询速度。MySQL 是业务数据库的重要组成部分，其提供的索引和分组排

序功能可以极大提升系统运行效率,故用于存放系统相关业务的数据库表,系统业务中涉及到的数据都保存在 MySQL。Elasticsearch 则主要存储的是音乐名称数据和歌手名称数据,主要为用户提供一个快速的正则化搜索功能,本节主要介绍业务数据库的存储。

4.4.1 概念模型设计

E-R 图可以更好地理解系统的数据结构,为后续的数据库设计提供了基础。在音乐推荐系统中,用户可以对音乐进行播放、收藏、点赞等操作,系统的 E-R 图如图 4-14 所示,由于实体属性过多,故 E-R 图中只展示了重要的属性。

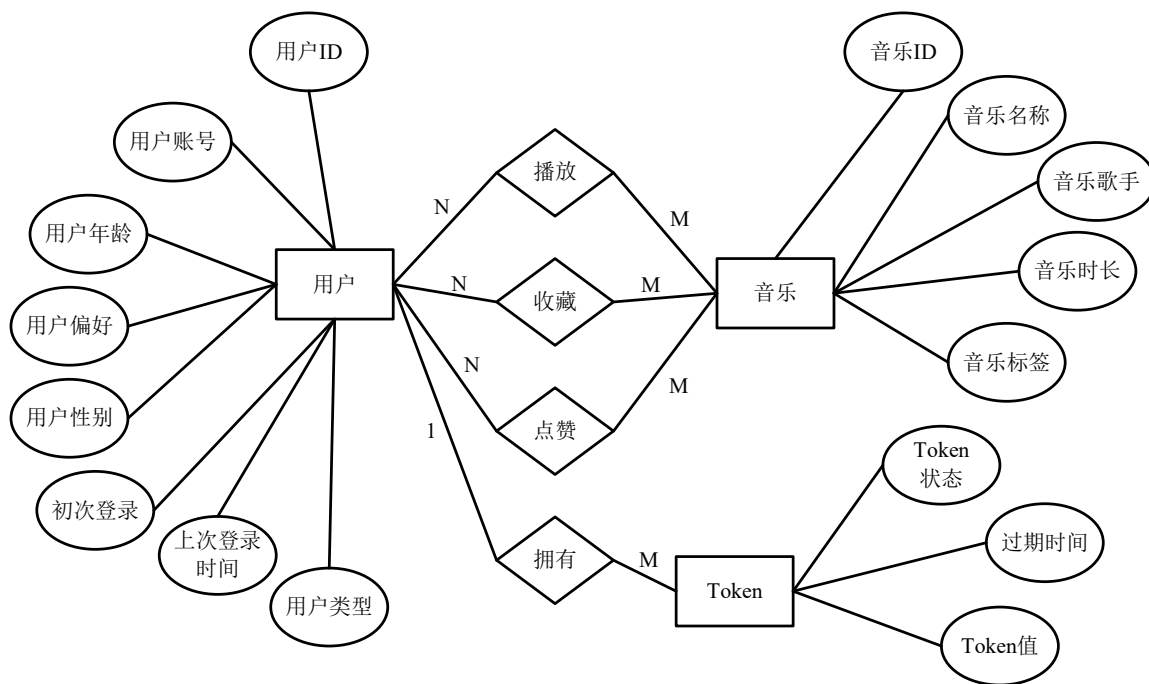


图 4-14 数据库 E-R 图

4.4.2 数据库表设计

而对于具体的数据库中的数据表设计,具体如下所示:

(1) 用户信息表

主要包含的数据有用户、账号、经 MD5 等其他非对称加密算法处理后的密码等重要信息、头像地址、昵称、手机号、用户性别、用户的偏好信息、用户类型。用户类型字段主要用于判断不同的用户,是用一个标志来区分系统中的不同用户类型。记

录用户上次登录时间的 login_time 字段,是用来判断用户是否是活跃用户。判断用户是否第一次登录的 first 字段,主要是为了解决冷启动问题,收集用户喜好而设置的,first 字段用 0 表示用户第一次登录,用 1 表示非首次登录,用户信息表结构如表 4-1 所示。

表 4-1 用户信息表

字段	数据类型	描述信息	主键	非空
id	int(11)	唯一 id	是	是
num	varchar(256)	账号	否	是
password	varchar(256)	密码	否	是
salt	varchar(256)	盐信息	否	是
avatar	varchar(256)	头像 url	否	否
phone_number	varchar(20)	手机号码	否	否
sex	varchar(10)	用户性别	否	否
country	varchar(10)	用户国家	否	否
age	int(10)	用户年龄	否	否
first	int(10)	标记初次登录	否	否
login_time	datetime	上次登录时间	否	否
user_type	int(2)	用户类型	否	否
like_type	varchar(50)	用户偏好信息	否	否

(2) 音乐信息表

音乐信息表作为存储从数据集中获取的音乐信息,是整个音乐推荐系统的核心表之一,包括了音乐名字、音乐导演、音乐主演、音乐国家、音乐类型、音乐标签等重要信息。其设计如表 4-2 所示。

表 4-2 音乐信息表

字段	数据类型	描述信息	主键	非空
id	int(11)	唯一 id	是	是
name	varchar(256)	音乐名称	否	是
singer	varchar(256)	音乐歌手	否	是

续表

lable	varchar(256)	音乐标签	否	否
play_time	int(64)	音乐时长	否	否
url	varchar(256)	音乐链接	否	否
cover	varchar(256)	封面链接	否	否

(3) Token 表

Token 是用户的登录令牌信息，是实现 JWT 的重要方式，每次登录都会创建一个新的 token 并分别存储在用户的浏览器 cookie 中以及服务器 Redis 的缓存中，其结构如表 4-3 所示。

表 4-3 Token 表

字段	数据类型	描述信息	主键	非空
id	int(11)	唯一 id	是	是
user_id	int(11)	用户的 id	否	是
token	varchar(128)	Token	否	是
expire	datetime	过期时间	否	否
status	int(11)	Token 状态	否	否

其中，Id 是 Token 的主键 ID，user_id 是持有该 token 的用户 id，expired 是 token 过期的日期，status 是辨别 token 是否已经失效。如果 Token 失效，用户必须再次登录系统。

(4) 音乐点赞表

音乐点赞表用来存储用户点赞过的音乐序列，其包含了用户 ID、音乐 ID、音乐类型和用户的点赞时间，其结构如表 4-4 所示。

表 4-4 音乐点赞表

字段	数据类型	描述信息	主键	非空
id	int(11)	唯一 id	是	是
user_id	int(11)	用户 id	否	是
music_id	int(11)	音乐 id	否	是

续表

type	varchar(256)	音乐类型	否	否
like_time	datetime	点赞时间	否	是

(5) 音乐收藏表

音乐收藏表用来存储用户收藏过的音乐序列，其包含了用户 ID、音乐 ID、音乐类型和用户的收藏时间，其结构如表 4-5 所示。

表 4-5 音乐收藏表

字段	数据类型	描述信息	主键	非空
id	int(11)	唯一 id	是	是
user_id	int(11)	用户 id	否	是
music_id	int(11)	音乐 id	否	是
favorite_time	datetime	收藏时间	否	是

(6) 音乐播放表

音乐播放表用来存储用户按一定规则过滤后的音乐播放序列，以便推荐算法使用，其包含了用户 ID、音乐 ID 和用户的播放时间，其结构如表 4-6 所示。

表 4-6 音乐播放表

字段	数据类型	描述信息	主键	非空
id	int(11)	唯一 id	是	是
user_id	int(11)	用户 id	否	是
music_id	int(11)	音乐 id	否	是
view_time	datetime	浏览时间	否	是

4.5 本章小结

本章分析了实时音乐推荐系统的非功能性需求和功能性需求，并根据需求分析引出了系统的整体架构设计、功能模块设计和数据库设计，其中重点介绍了音乐推荐模块和实时推荐流程的设计，为后续系统的实现奠定了基础。

5 个性化实时音乐推荐系统实现

本章将介绍系统开发和部署环境以及系统的具体实现，包括用户和系统管理模块、音乐推荐和搜索模块、数据可视化和交互模块的实现。对于系统开发过程中关键技术的实现则穿插在功能实现中进行介绍。

5.1 音乐推荐系统开发和部署环境

系统的开发是在个人笔记本电脑上进行的，电脑的处理器的 AMD Ryzen 7 5800X3D with Precision Boost 2，内存为 32GB。使用到的开发工具和对应版本如表 5-1 所示。

表 5-1 开发工具

开发工具	版本
IDE	IDEA 2022.2.3 (Community Edition)
JDK	1.8.0_221
Maven	3.8.1
Spring Cloud	2022.0.1
MyBatis Plus	3.0

为了实现数据计算的并行化，并满足上述非功能需求中的高可靠和高可用性要求，使用了三台 Linux 服务器进行部署。

各服务器上部署的软件版本以及在集群中充当的角色如表 5-2 所示。

表 5-2 集群环境工具

工具	版本	服务器集群		
		Linux 0	Linux 1	Linux 2
Flink 集群	flink-1.1.6-bin-hadoop2.7.tgz	Flink Master	Flink Slave	Flink Slave
JDK	jdk-8u208-linux-x64.tar.gz	JDK 0	JDK 1	JDK 2
Tomcat	apache-tomcat-8.3.31.tar.gz	Tomcat 0	Tomcat 1	Tomcat 2

续表

Zookeeper 集群	zookeeper- 3.6.10.tar.gz	Zk Follower	Zk Follower	Zk Leader
Filebeat	filebeat-8.7.1-linux- x86_64.tar.gz	Filebeat 0	Filebeat 1	Filebeat 2
Elasticsearch	elasticsearch- 5.6.3.tar.gz	ES 0	ES 1	ES 2
Scala	scala-2.12.8.zip	Scala 0	Scala 1	Scala 2
Kafka 集群	kafka_2.12-2.2.0.tgz	Kafka Broker 1	Kafka Broker 2	Kafka Broker 3
Azkaban	azkaban-solo-server- 3.36.0.tar.gz	Azkaban 0	Azkaban 1	Azkaban 2
MySQL	MySQL-8.0.31-linux- glibc2.12- x86_64.tar.xz	MySQL 0	MySQL 1	MySQL 2
Redis	redis-6.0	Redis 0	Redis 1	Redis 2

5.2 音乐推荐系统功能实现

在对推荐系统的架构进行设计后，本节主要通过软件工程的方式介绍和实现业务逻辑并辅以系统实现界面进行展示。系统基于 Spring Cloud 分布式微服务框架实现，故在系统设计之初，就将系统的各个业务模块独立成子系统进行设计和整合。Spring Cloud 将每个模块独立成单独的服务部署在不同的集群机器上，各自通过 Zookeeper 注册中心进行服务注册和发现，在不同模块之间使用 RPC 调用方法进行远程调用。下面主要对系统的核心模块的实现作简要介绍。

5.2.1 用户和系统管理模块实现

用户和系统管理模块包括系统和音乐管理微模块和用户管理微模块。系统管理微模块作为整个系统权限最高的模块，只有拥有管理员角色的用户方可进入本模块。其主要系统有系统监控和告警查看以及音乐的信息管理两项功能，监控是通过服务定期轮询各大组件提供的 API 接口来获取组件的运行状态并收集存储于数据库中，方便展示给管理员进行数据分析和使用。另外，管理员也可设置相关的告警阈值来对某个指标进行自动化的告警，如图 5-1 所示。

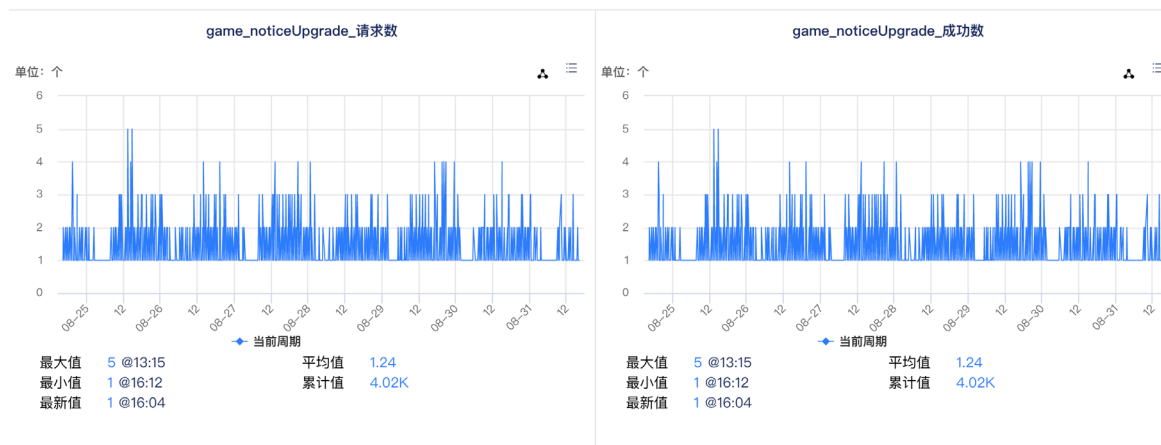


图 5-1 系统状态监控实现效果图

音乐管理微模块是管理员管理音乐的模块，其基本原理就是对音乐数据库表中的信息进行增加、删除、修改和查询，如图 5-2 所示展示的是音乐管理实现效果，管理员通过点击左侧导航栏中的音乐管理菜单进入该页面，整个页面被分为上下三个部分，最上面是音乐搜索功能，管理员可通过音乐名称和歌手对音乐信息表进行搜索。页面中间是音乐信息表的展示和操作模块，包括曲库中的音乐名称、歌手、音乐标签、时长、播放链接等，管理员可通过相应操作添加音乐至曲库，查看音乐详情和删除音乐。页面最下部分是页码导航栏。

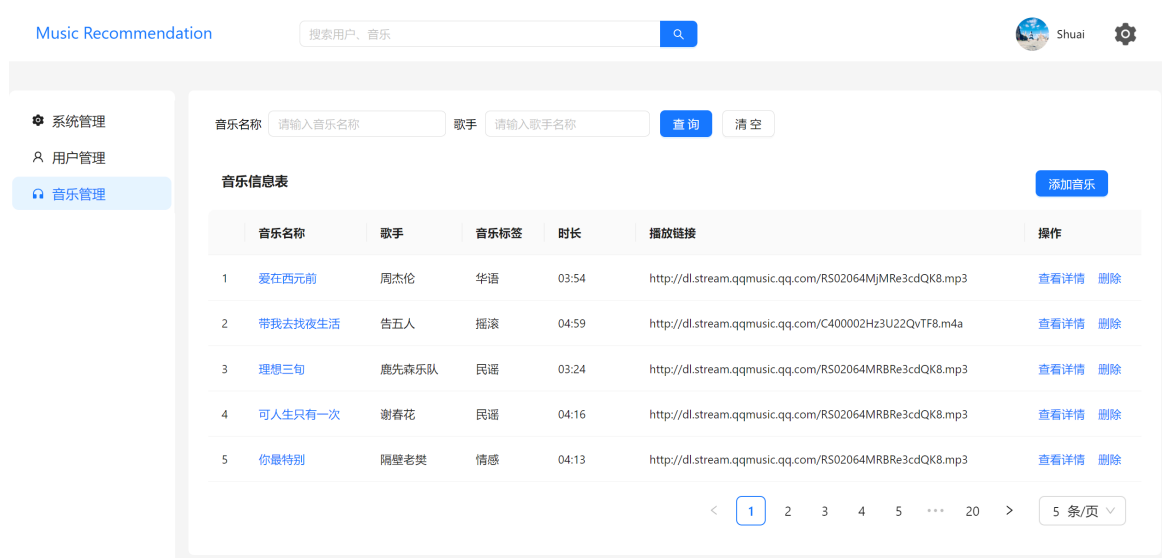


图 5-2 音乐管理实现效果图

用户管理模块的功能和实现类似，都是对数据库表中的数据进行修改查询和增加，区别在于音乐管理微模块是管理员管理音乐的模块，用户管理模块管理用户，在此不作赘述。

5.2.2 音乐推荐和搜索模块实现

音乐推荐和搜索模块是音乐推荐系统中的核心模块，主要包括音乐推荐微模块和音乐搜索微模块，其中音乐推荐包含实时推荐、离线推荐和热门推荐三个功能。音乐搜索包括分类搜索和正则化模糊搜索功能。

今日推荐功能即离线推荐功能，其实现效果如图 5-3 所示。页面的下方是本功能的主要展示区域，即今日推荐列表区域，展示了今日推荐歌曲列表中歌曲的相关信息，包括歌曲名称，歌曲对应歌手，歌曲时长等重要信息。对于推荐列表中的每首推荐歌曲，可通过点击的方式进行播放、收藏、点赞等操作。

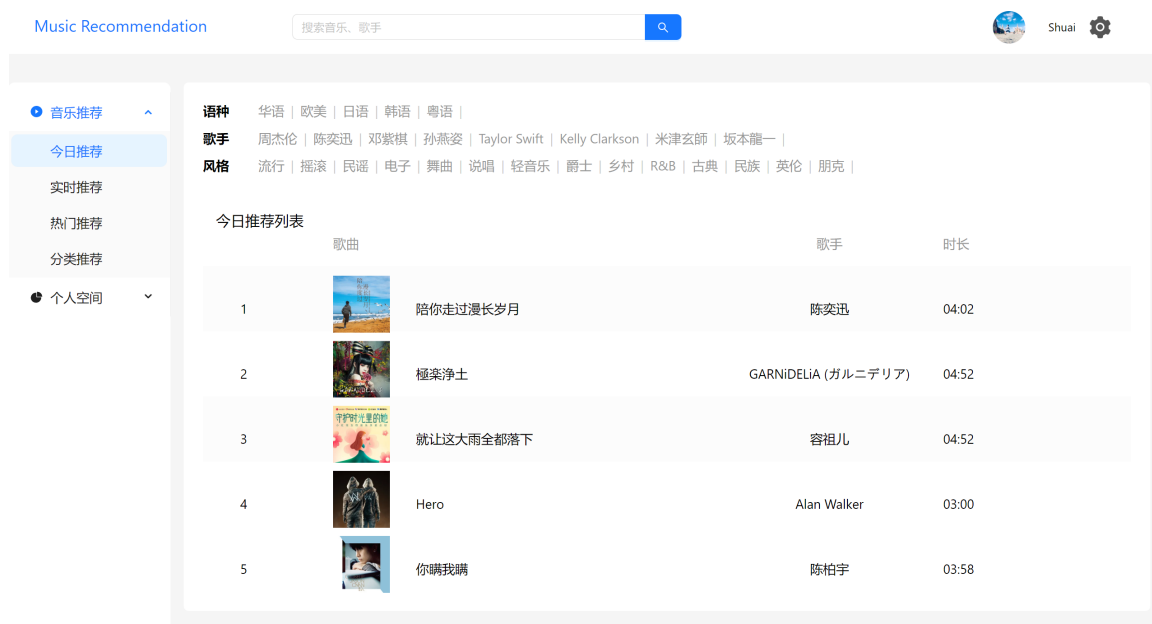


图 5-3 今日推荐功能实现效果图

上述功能的实现依赖于用户行为信息的收集。在系统中，因为音乐的相关信息主要来自于数据集，所以只需要收集用户历史行为数据，用户历史行为数据的收集主要有两个部分，分别是用户播放信息和用户使用信息。音乐的推荐微模块首先会对用户的登录情况做判断，并且对用户是否是首次登录进行判断，从而规避推荐系统常见的

冷启动问题。在系统中，由于音乐的数据和分类主要来自于数据集数据，系统也有足够的数据集来进行训练，所以系统的冷启动问题主要是指当系统中增添新用户，因为新用户没有历史使用记录，没法进行有效地推送音乐给新用户的问题。

论文旨在通过获取用户的偏好来解决新用户的冷启动问题，系统通过在用户数据库中新增一个首次登录标志位来确定这是否是用户们的第一次登录，当用户第一次登录时，会被提示去选择其所感兴趣的音乐类型，此音乐类型会被储存起来，以便在随后的推荐算法模型中使用，也就是让用户在注册完成之后进入到音乐喜好选择页面来进行感兴趣音乐类型的选择，使得推荐系统能够快速地对新用户进行合理而高效的推荐。

根据用户在注册时选择的音乐类型，音乐推荐模块会在系统中搜索符合所选类型的音乐，根据音乐播放量、喜欢度和收藏数据进行加权，最后根据加权分数的大小将音乐从高到低排序。其加权公式如式(5-1)所示。

$$Score = \frac{\frac{v}{V_c} + \frac{l}{L_c} + \frac{r}{R_c}}{3} \quad (5-1)$$

其中， $Score$ 是最终的加权分数， V_c 是所有音乐的播放量总和的均值， v 是当前音乐播放量， L_c 是所有音乐的喜欢度总和的均值， l 是当前音乐喜欢度， R_c 是所有音乐收藏量的均值， r 是当前音乐收藏量。这是用户在选择音乐类型之后的首次个性化推荐功能。

如果用户没有选择音乐类型，系统会根据统计学上的非个性化推荐功能，推荐播放最高的热门音乐。这种方法在没有用户兴趣数据信息的情况下，尽最大程度为用户提供质量较高而热门的音乐，以改善用户的体验，提高了对此音乐平台的粘性和留存度，进而使用户产生更多的使用记录，不断地为用户提供更加精确的算法推荐，从而形成良性循环。

播放最高的热门音乐的推荐即热门推荐功能，其实现效果如图 5-4 所示，点击左侧的音乐推荐导航栏中的热门推荐选项进入该页面，该区域展示了热门推荐歌曲列表中歌曲的相关信息，包括歌曲名称，歌曲对应歌手，歌曲时长等重要信息。对于推荐列表中的每首推荐歌曲，可通过点击的方式进行播放、收藏、点赞等操作。

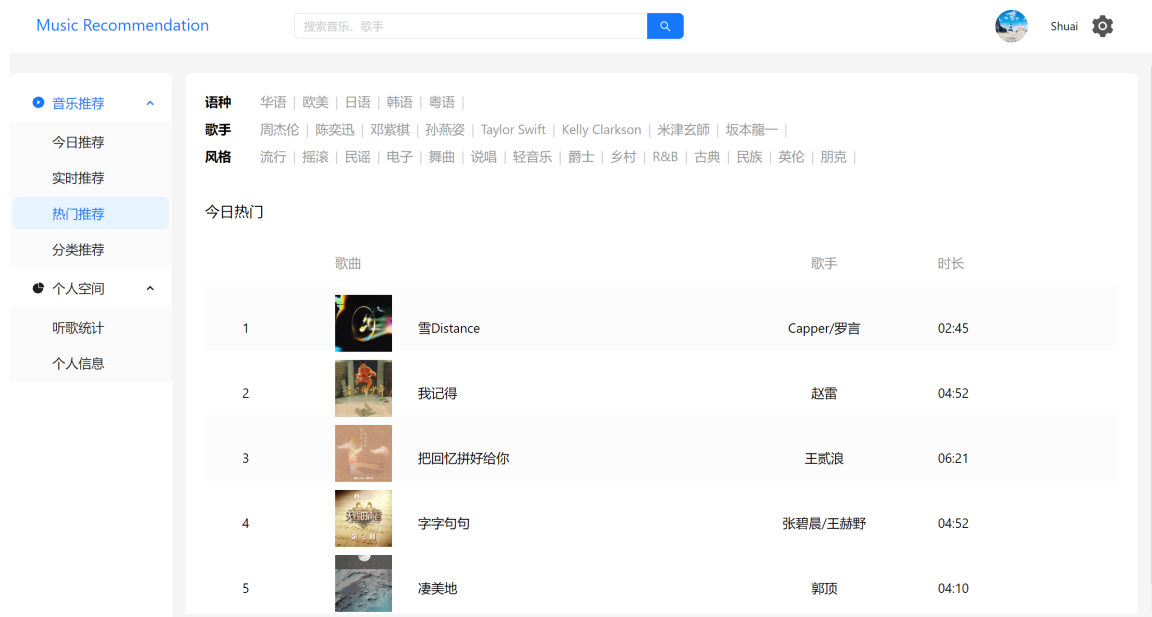


图 5-4 热门推荐功能实现效果图

分类推荐功能实现效果如图 5-5 所示，点击左侧的音乐推荐导航栏中的分类推荐选项进入该页面，用户可在本区域的上方对音乐的歌手、语种和风格进行选择，系统会根据存储的数据结构，在 SQL 层面进行分组和过滤，最终通过前端的界面进行展示。

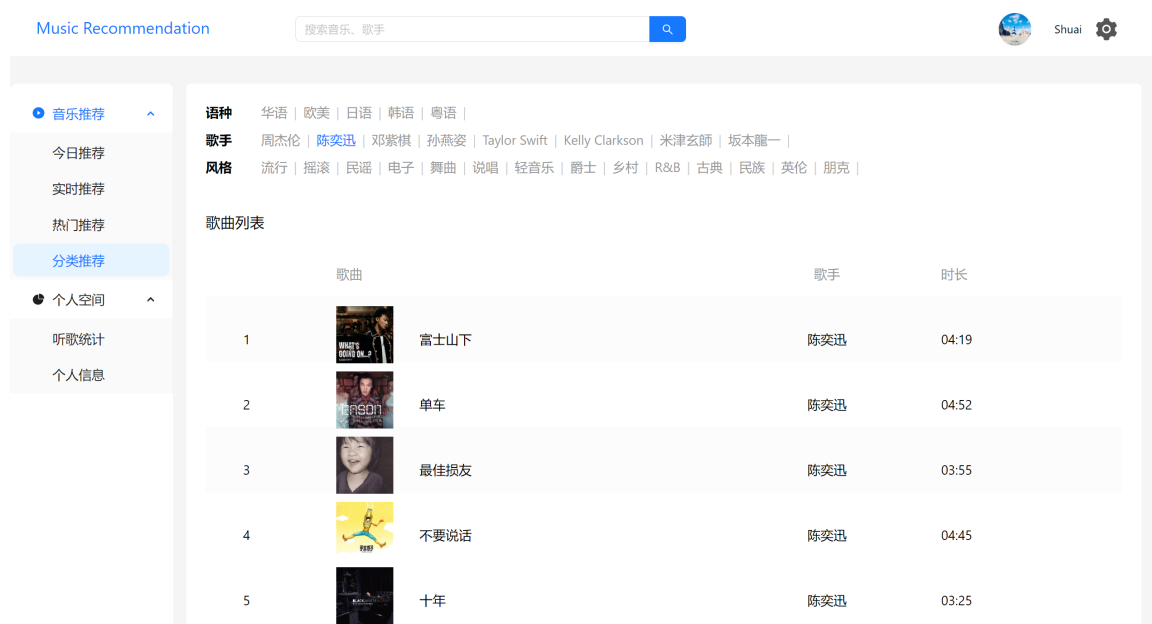


图 5-5 分类推荐功能实现效果图

音乐搜索功能作为基础功能之一，用户可通过系统中的全局搜索和个人信息状态栏中的搜索框进行使用。在搜索功能设计之初，考虑到用户会通过歌曲名称和歌手名称进行搜索，所以两个数据都存储 Elasticsearch 中以拓展用户的搜索方式，其效果如图 5-6 所示。



图 5-6 音乐搜索功能实现效果图

5.2.3 数据可视化和交互模块实现

数据可视化微模块作为用户个人数据的展示页面，是用户历史行为数据存储的展示。主要包括用户近期收听的音乐、用户收藏音乐和用户喜欢的音乐功能组成。其中，用户的近期收听音乐存储在历史播放音乐表中，而历史收藏存储于收藏表中，当用户进行首次查询时，会调用 MySQL 数据库的 select 方法，这些数据会在用户登录后缓存于 Redis 中，并于登录 token 设置为相同的过期时间，方便用户在登录态的情况下，快速地查看，并减少用户的频繁操作对 MySQL 数据库的读写影响。用户点击导航栏中的个人空间列表选项中的听歌统计子菜单进入该页面，整个页面被分为音乐品味和数据可视化两部分。左侧是当前登录用户的音乐品味，包括“我喜欢的音乐”、“我近期收听的音乐”、“我收藏的音乐”三大列表的数据展示，用户可通过查看

列表按钮具体查看。而右侧是当前用户的听歌数据可视化，其包含了用户近一个月内播放次数最多的歌曲对应的歌手的数据可视化功能，系统采用了可区分颜色的饼状图进行展示，其实现效果如图 5-7 所示。

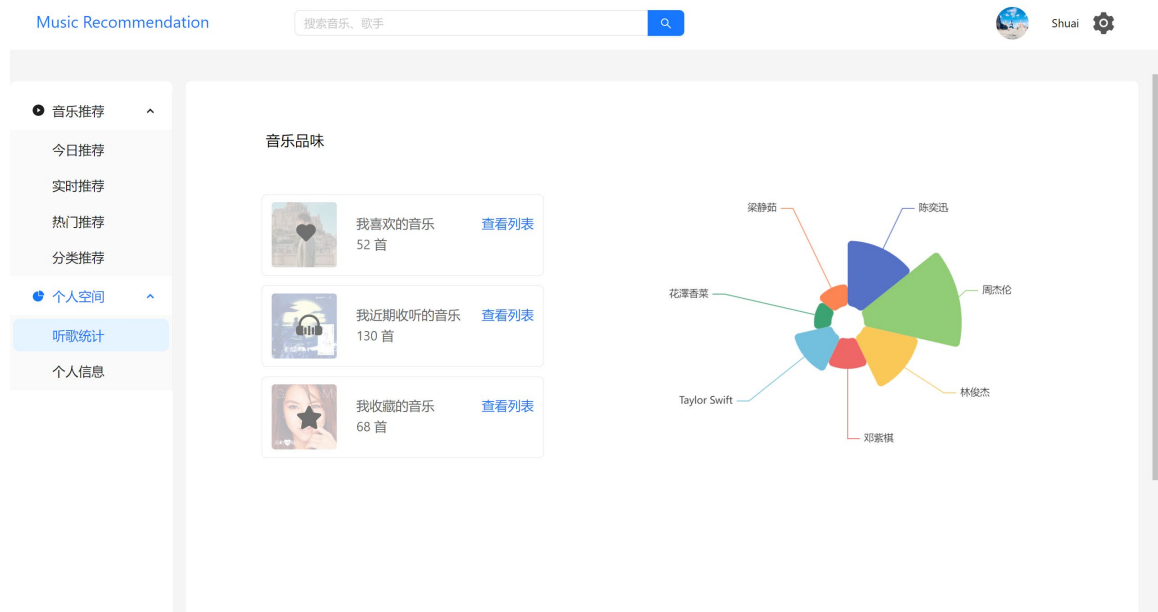


图 5-7 听歌统计数据可视化实现效果

个人信息微模块中最首要的是保证用户的信息安全。用户在进行注册时，系统会保证用户输入的用户名的唯一性，此功能通过数据库的 UNIQUE 索引实现。另外，在用户输入密码时，前端会进行遮蔽显示，后端会进行 MD5+Salt 的加密存储，保证用户的信息安全。在用户进行登录时，系统会使用与注册时同样的加密算法进行运算，并将运算得到的数据与数据库中已加密的数据进行对比，从而确保准确性和安全性。另外，系统每次都会通过随机算法生成一个 Token 下发给用户，并存储于后端数据库中，以便实现登录有效期和会话超时等功能。

5.3 本章小结

本章首先介绍了系统的开发和部署环境，并对系统三个模块中的关键功能的具体实现和关键技术实现作了简要的介绍，其中重点介绍了实时性系统的实现，以便后续的系统测试。

6 个性化实时音乐推荐系统测试

本章将对音乐推荐系统的测试目的、测试方法和测试环境作简要介绍，并对系统进行功能测试和性能测试，最后对测试结果进行分析和总结。

6.1 音乐推荐系统测试目的和方法

系统测试是在系统开发完成后重要的一个环节，是保证系统可用性和完整性是否达到预期目标的检验方法，有以下两个步骤：

（1）功能性测试

功能性测试是系统测试的第一步，主要是测试系统在设计之初的各项功能是否达到了预期的设计要求。

（2）性能/压力测试

性能测试和压力测试，主要是模拟在高并发，用户量大的情况下以及在用户数据多的情况下，系统是否能保持原有响应速度，和在高峰期时系统的稳定性。本系统最初的设计目标便是个性化实时音乐推荐系统，因此系统的实时性和性能测试是本章节的重点测试环节，其中设计和对比了系统在用户数和用户音乐播放数两个指标的系统响应时间。

6.2 音乐推荐系统测试环境

系统使用的测试软件如表 6-1 所示。

表 6-1 系统测试软件表

测试软件	版本及名称
客户端测试软件	Microsoft Edge 109.0.1518.70 (64 位)
服务端测试软件	Apache JMeter 5.5
数据库可视化软件	WorkBench 8.0
接口测试软件	Postman v10.8.0
缓存可视化软件	Another Redis Desktop Manager v1.5.9

系统部署在服务器上，具体配置如表 6-2 所示。

表 6-2 服务器配置详情表

名称	配置
虚拟机的系统	Linux
处理器的个数	8
服务器系统版本	Centos 7
CPU 的核心数	16
内存大小	16G

系统的功能测试主要通过微软的 Edge 浏览器进行，通过模拟管理员和用户的行为来进行测试，系统的性能测试主要是通过 Linux 系统下的 ab 压测命令、Postman 压测工具、JMeter 压测工具和接口测试命令 curl 一起进行。

6.3 音乐推荐系统测试内容与结果分析

本节将通过功能测试和性能测试对系统进行全面的测试，并对测试结果进行分析和总结。

6.3.1 系统功能测试

通过使用 Linux Shell 和 Python 编写的脚本以及模拟用户使用场景进行功能测试，主要测试系统功能的完善性和可用性是否达到预期目标，每个模块的测试内容如下：

（1）用户和系统管理模块测试

系统管理模块作为整个系统权限最高的模块，只有拥有管理员角色的用户方可进入本模块，其主要有监控和告警两项功能，因此测试的重点主要在权限管理和监控和告警上。包括普通用户是否能正常被系统告知权限不足，管理员是否能正常的进入功能模块，监控功能是否正常显示，以及告警功能的测试。

（2）音乐推荐和搜索模块测试

音乐推荐功能主要是通过离线推荐算法和实时推荐算法进行音乐的排序和推荐。测试的主要内容包括实时推荐算法和离线推荐算法的准确性、时效性以及算法在大

量数据情况下的性能。另外，根据用户的音乐播放次数得出的 Top-N 排序列表，需要测试列表是否符合数据库数据。

音乐搜索功能作为基础功能之一，是通过 Elasticsearch 全文搜索引擎来进行实现的，这样用户在使用搜索功能时，可以使用模糊搜索。因此本模块的测试重点在于 Elasticsearch 是否可以正常使用，Elasticsearch 中的数据增删改查是否正常，以及用户在使用模糊搜索功能是否正常。

（3）数据可视化和交互模块测试

数据可视化和交互模块包括数据可视化、账号信息管理、音乐信息管理三大功能。数据可视化作为用户个人数据的展示页面，是用户历史行为数据存储的展示，主要由历史音乐播放、历史音乐点赞等功能组成。因此本模块的测试重点在于，多个用户是否能正常进入本用户的用户空间页面，以及用户空间中的历史音乐播放、历史音乐点赞等历史用户数据是否无误等。

信息管理微模块是对使用系统的用户的个人信息的数据管理模块，包括用户登录注册，密码找回等基本功能，所以此模块的测试主要是对系统登录功能、用户注册功能、密码找回功能进行测试。测试过程主要通过点击相关功能按钮是否能出现对应的功能页面，以及数据库中的数据是否新增和修改，数据库中的数据在并发事务的情况下是否正确等情况。经过上述的功能测试分析，每个模块的测试结果如表 6-3 所示。

表 6-3 系统功能测试结果表

功能模块	测试结果
用户和系统管理模块	符合预期
音乐推荐和搜索模块	符合预期
数据可视化和交互模块	符合预期

6.3.2 系统性能测试

在本推荐系统中，实时性和性能表现也同样是非常重要的指标。Redis 主要是用作内存缓存，MySQL 主要用作业务数据库表的存储，而 Elasticsearch 主要用于搜索功能，其中主要存放的是推荐模块所使用到的用户近期若干音乐播放数据，这里为了

对比未使用实时数据处理框架和基于 Flink、Spark Streaming 两个大数据计算框架下算法的数据处理执行效率差别，从 Last.fm 数据集中随机选取了不同数量级的用户播放数据并写入到 Redis 和 MySQL 中。实验结果如图 6-1 所示。

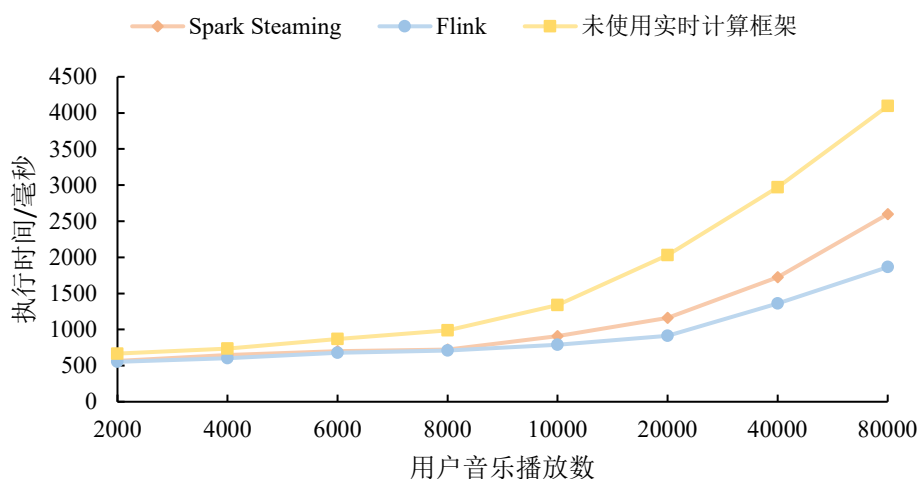


图 6-1 不同音乐播放数和实时计算框架下推荐数据处理的执行时间

图 6-2 则是固定 Redis 内存缓存中的用户对各项音乐评分数据的数量为 2000，在 Flink 和 Spark Streaming 两个大数据计算框架的算法数据预处理执行效率的情况，即对用户规模量的影响进行测试。

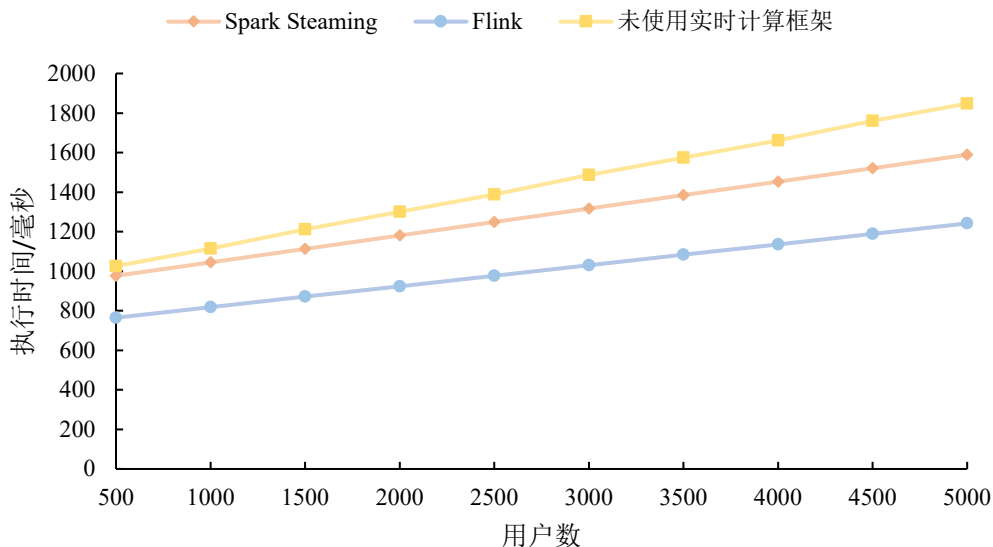


图 6-2 不同用户数下推荐数据处理的执行时间

在图 6-1 和 6-2 中，可以明显看出，推荐算法的数据处理在 Flink 流式大数据计算框架时，效率都比基于 Spark Streaming 计算框架更高。并且在数据规模越来越大时，Flink 的计算耗时增长得也比较缓慢，相比于 Spark Streaming 具有更大的优势。因此，论文推荐模块的大数据处理框架选择 Flink 会更加合适，也证实了 Flink 对系统实时计算效率的提升效果，符合实时推荐系统的设计要求。

6.4 本章小结

本章首先介绍了系统的测试目的、测试方法和测试环境，随后对系统进行了功能测试和性能测试。测试结果表明，系统功能正常，在 Flink 上的运行性能较好，系统达到了实时性的设计预期。

7 总结与展望

7.1 全文总结

在信息时代,人们在享受互联网和大数据带来便利的同时,也越来越难以从大量的信息数据中寻找自身感兴趣或对自身有价值的信息,从最开始出现的搜索引擎、知识图谱,再到现在的推荐系统,无一不是为了过滤大量的无效信息。搜索引擎作为主动获取信息的解决方案,已经十分成熟和完善。而对于被动获取信息的推荐系统,也在工业界和学术界的研究下愈加完善,根据前人的研究和不同类型的神经网络的特点,论文设计并实现了一种基于卷积神经网络和带自注意力机制的长短期记忆网络推荐算法的音乐推荐系统,解决了推荐系统中个性化特征提取不足、实时性低的问题,论文的主要工作和创新点如下:

(1) 设计了基于 CNN-LSTM-SA 算法的个性化推荐模型,解决了推荐系统中个性化特征提取不足的问题。通过分析目前主流的基于协同过滤算法和深度神经网络模型算法的原理和优缺点,确定论文采用卷积神经网络(CNN)和带自注意力机制的长短期记忆网络(LSTM-SA)相结合的推荐算法作为系统的推荐模块,针对 CNN 擅长处理稀疏数据而 LSTM 擅长处理时序数据的特性,论文将音乐特征数据和用户行为序列数据经过 Flink 计算预处理后分别输入到 CNN 和 LSTM 进行处理,以求更好地学习到用户与音乐之间的个性化偏好信息。最后在数据集上验证模型,并与多种推荐算法模型进行对比实验,实验结果表明,本模型相较于其他深度学习模型,拥有更优秀的效果。

(2) 设计并实现了个性化实时音乐推荐系统,提升了音乐推荐系统的实时性。系统选用了当下大数据方面前卫的工具 Flink 实时计算框架,以及 Java EE 开发框架工具 Spring Cloud、MySQL、Kafka、Filebeat、Redis、Elasticsearch 等组件对各核心功能模块进行了详细设计与实现,系统由音乐推荐和搜索模块、数据可视化和交互模块、用户和系统管理模块组成。在系统中,基于 CNN-LSTM-SA 推荐算法构建了具备个性化实时推荐能力和离线推荐能力的音乐推荐模块;基于 B/S 架构实现了交互性良好的系统可视化和异常告警能力;使用 Flink 实时计算框架、Kafka 消息队列和

Redis 缓存提升了异构数据的实时处理能力和实时推荐能力。

(3)对 CNN-LSTM-SA 算法的个性化推荐模型和个性化实时音乐推荐系统进行测试。实验结果表明, CNN-LSTM-SA 算法在 Top 50 命中率上达到了 86.3%, 解决了个性化特征提取不足的问题, 满足个性化推荐的要求。基于 CNN-LSTM-SA 推荐算法的个性化实时音乐推荐系统在系统功能测试表现良好, 在系统性能测试上延迟可控制在秒级, 解决了实时性低的问题, 符合实时推荐系统的设计要求, 具有较好的实际意义和实用价值。

综上, 论文提出的基于卷积神经网络和带自注意力机制的长短期记忆网络推荐算法的个性化实时音乐推荐系统能够有效地进行音乐推荐, 使用户可以在大量的音乐中找到自己的喜好。同时, 系统能有效地提高计算效率和稳定性, 满足了实时性的要求。

7.2 研究展望

虽然所使用的基于卷积神经网络和长短期记忆网络推荐算法的个性化实时推荐系统取得了不错的效果, 但系统还是有一些不足之处需要进行考虑和完善。

(1) 适用场景问题

论文的推荐系统仅考虑到应用于音乐推荐, 而目前推荐系统的场景已层出不穷, 如 MOBA 游戏中的英雄推荐、咨询系统中的专家推荐、电商系统的商品推荐等等。将论文的推荐系统改进并适用于其他推荐场景也是今后需要探索和研究的问题。

(2) 长短期兴趣混杂问题

论文采用卷积神经网络与循环神经网络相结合的算法虽然能感知用户的时间维度上的兴趣变化, 提升了推荐算法的效果。但用户的长短期兴趣存在差异, 长期兴趣相对稳定, 短期兴趣动态变化, 论文暂时缺乏长短期兴趣的标签。采用了单一的循环神经网络对用户兴趣建模, 则容易将两个方面混杂在一起, 导致性能提升效果没有发挥到极致, 这也是后续需要探索和研究的问题。

参考文献

- [1] 中国互联网络信息中心. "数字"点亮美好生活——透视第 50 次《中国互联网络发展状况统计报告》. 信息系统工程, 2022(10): 4-5
- [2] 张安琪, 李元旭. 互联网知识共享平台信息过载效应与弱化机制——基于知乎的案例研究. 情报科学, 2020, 38(1): 24-29
- [3] Bink A B, Corrigan P. The impact of mental health information overload on community education programs to enhance mental health-care seeking. Journal of public mental health, 2022(2): 174-178
- [4] 杨晨. 淘宝搜索引擎优化技巧研究. 电子商务, 2022, 3(1): 39-40
- [5] 张洪辰. 新浪微博数据抓取——高级搜索. 信息与电脑: 理论版, 2013(11): 54-55
- [6] Sanchez M G. The influence of Google search index on stock markets: an analysis of causality in-mean and variance. Review of Behavioral Finance, 2021, 13(2): 202-226
- [7] Wang S , Hu L , Wang Y , et al. Graph Learning based Recommender Systems: A Review. in: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21), Montreal, Canada, 19-26 Aug. 2021, ACM, 2021: 4644-4652
- [8] 李爽. 面向准确性和多样性的个性化推荐算法研究[博士学位论文]. 北京: 清华大学, 2018
- [9] 朱义奎, 黄佳豪, 蔡亮. 基于 Spark 机器学习的电商推荐系统的设计与实现. 现代商贸工业, 2021, 42(S01): 52-54
- [10] 赵辰玮, 刘韬, 都海虹. 算法视域下抖音短视频平台视频推荐模式研究. 出版广角, 2019(18): 76-78
- [11] Quijano-Sanchez L, Recio-Garcia J A, Diaz-Agudo B. HappyMovie: A Facebook Application for Recommending Movies to Groups. in: 2011 IEEE 23rd International Conference on Tools with Artificial Intelligence (ICTAI), Boca Raton, FL, USA, 7-9 Nov. 2011, IEEE, 2011: 239-244
- [12] 高明, 金澈清, 钱卫宁,等. 面向微博系统的实时个性化推荐. 计算机学报, 2014,

37(4): 963-975

- [13] Koren Y. Factorization meets the neighborhood: A multifaceted collaborative filtering model. in: 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Las Vegas, Nevada, USA, Aug. 24-27 2008, ACM, 2008: 426-434
- [14] 刘红霞. 基于协同过滤技术的推荐系统综述. 信息安全与技术, 2016(3): 24-26
- [15] Song Z, Cong L, Li M, et al. Alleviating the sparsity problem of collaborative filtering using rough set. COMPEL: The International Journal for Computation and Mathematics in Electrical and Electronic Engineering, 2013, 32(2): 516-530
- [16] Zhou G, Song C, Zhu X, et al. Deep Interest Network for Click-Through Rate Prediction. in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, London, UK, Aug. 19-23 2018, ACM, 2018: 1059-1068
- [17] Kuzelewska, Urszula. Clustering Algorithms in Hybrid Recommender System on MovieLens Data. Studies in Logic, 2014, 37(1): 125-139
- [18] Linden, G, Smith, et al. Amazon.com recommendations: item-to-item collaborative filtering. Internet Computing IEEE, 2003, 7(1): 76-80
- [19] Program W. Proceedings of KDD Cup and Workshop. Web Semantics Science Services, 2020(04): 67-71
- [20] Bell R M, Koren Y. Lessons from the Netflix prize challenge. ACM SIGKDD Explorations Newsletter, 2007, 9(2): 75-79
- [21] Koren Y, Bell R, Volinsky C. Matrix factorization techniques for recommender systems. IEEE Computer Journal, 2009, 42(8): 30-37
- [22] Hidasi B, Quadrana M, Karatzoglou A, et al. Parallel Recurrent Neural Network Architectures for Feature-rich Session-based Recommendations, in: Proceedings of the 10th ACM Conference on Recommender Systems, Boston Massachusetts, USA, 15-19 Sep. 2016, ACM, 2016: 241-248
- [23] 张锋, 常会友. 使用BP神经网络缓解协同过滤推荐算法的稀疏性问题. 计算机研究与发展, 2006, 43(4): 667-672
- [24] Hinton G E, Osindero S, Teh Y W. A Fast Learning Algorithm for Deep Belief Nets. Neural computation, 2006, 18(7): 1527-1554

- [25] Swersky K, Bo C, Marlin B, et al. A tutorial on stochastic approximation algorithms for training Restricted Boltzmann Machines and Deep Belief Nets, in: 2010 Information Theory and Applications Workshop (ITA), La Jolla, USA, 31 Jan.-05 Feb. 2010, IEEE, 2010: 1-10
- [26] Cinyol F, Mutlu H E, Baysal U. Classification of lung sounds with convolutional neural network, in: 2017 21st National Biomedical Engineering Meeting (BIYOMUT), Istanbul, Turkey, 24 Nov.-26 Dec. 2017, IEEE, 2017: i-iv
- [27] Covington P, Adams J, Sargin E. Deep Neural Networks for YouTube Recommendations, in: Proceedings of the 10th ACM Conference on Recommender Systems, Boston Massachusetts, USA, 15-19 Sep. 2016, ACM, 2016: 191-198
- [28] Dacrema M F, Cremonesi P, Jannach D. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches, in: Proceedings of the 13th ACM Conference on Recommender Systems, Copenhagen, Denmark, 16-20 Sep. 2019, ACM, 2019: 101-109
- [29] Nelson K P, Mitani A A, Edwards D. Evaluating the effects of rater and subject factors on measures of association. Biometrical Journal, 2018, 24(4): 107-112
- [30] Shardanand U, Maes P. Social information filtering: Algorithms for automating “word of mouth”, in: Proceedings of the SIGCHI conference on Human factors in computing systems, Denver, USA, 7-11 May. 1995, ACM, 1995: 210-217
- [31] McFee B, Bertin-Mahieux T, Ellis D P W, et al. The million song dataset challenge, in: Proceedings of the 21st International Conference on World Wide Web, Lyon, France, 16-20 Apr. 2012, ACM, 2012: 909-916
- [32] Aioli F. Efficient top-n recommendation for very large scale binary rated datasets, in: Proceedings of the 7th ACM conference on Recommender systems, Hong Kong, China, 12-16 Oct. 2013, ACM, 2013: 273-280
- [33] Bogdanov D, Haro M, Fuhrmann F, et al. Semantic audio content-based music recommendation and visualization based on user preference examples. Information Processing & Management, 2013, 49(1): 13-33
- [34] Hyung Z, Lee K, Lee K. Music recommendation using text analysis on song requests to radio stations. Expert Systems with Applications, 2014, 41(5): 2608-2618
- [35] Horsburgh B, Craw S, Massie S. Learning pseudo-tags to augment sparse tagging in

- hybrid music recommender systems. *Artificial Intelligence*, 2015, 219: 25-39
- [36] Oramas S, Nieto O, Sordo M, et al. A deep multimodal approach for cold-start music recommendation, in: *Proceedings of the 2nd workshop on deep learning for recommender systems*, New York, United States, 27 Aug. 2017, ACM, 2017: 32-37
- [37] Kowald D, Schedl M, Lex E. The unfairness of popularity bias in music recommendation: A reproducibility study, in: *Advances in Information Retrieval: 42nd European Conference on IR Research, ECIR 2020, Lisbon, Portugal, 14–17 Apr. 2020*, Springer International Publishing, 2020: 35-42
- [38] Chen K, Liang B, Ma X, et al. Learning audio embeddings with user listening data for content-based music recommendation, in: *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Toronto, Canada, 06-11 June 2021, IEEE, 2021: 3015-3019
- [39] Park J, Kim D, Kim D. Item-based Variational Auto-encoder for Fair Music Recommendation. *arXiv preprint arXiv:2211.01333*, 2022
- [40] Dean J, Ghemawat S. MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, 2008, 51(1): 107-113
- [41] Ghemawat S, Gobioff H, Leung S T. The Google File System, in: *Proceedings of the nineteenth ACM symposium on Operating systems principles*, Bolton Landing NY, USA, 19-22 Oct. 2003, ACM, 2003: 29-43
- [42] Chang F, Dean J, Ghemawat S, et al. Bigtable: A Distributed Storage System for Structured Data. *Acm Transactions on Computer Systems*, 2008, 26(2): 1-26
- [43] 何思佑, 王亚强. Hadoop 研究及挑战综述. *信息通信*, 2018, 2(3): 290-291
- [44] Lathar P, Srinivasa K G. A Study on the Performance and Scalability of Apache Flink Over Hadoop MapReduce. *International Journal of Fog Computing*, 2019, 2(1): 61-73
- [45] D García-Gil, S Ramírez-Gallego, S García, et al. A comparison on scalability for batch big data processing on Apache Spark and Apache Flink. *Big Data Analytics*, 2017, 82(3): 323-356
- [46] Sarwar B, Karypis G, Konstan J, et al. Item-based Collaborative Filtering Recommendation Algorithms. *ACM*, 2001, 3(1): 769-792

- [47] Cui L, Huang W, Qiao Y, et al. A novel context-aware recommendation algorithm with two-level SVD in social networks. *Future Generation Computer Systems*, 2017, 86(SEP.): 1459-1470
- [48] 王成, 朱志刚, 张玉侠, 等. 基于用户的协同过滤算法的推荐效率和个性化改进. *小型微型计算机系统*, 2016, 37(3): 428-432
- [49] LeCun Y, Boser B, Denker J S, et al. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1989, 1(4): 541-551
- [50] 张文凯. 基于深度学习的图像分类及图像集总结方法研究[博士学位论文]. 北京: 中国科学院大学, 2018
- [51] 刘一宁. 基于机器学习的深海水声被动定位研究[博士学位论文]. 北京: 中国科学院大学, 2022
- [52] Joulin A, Grave E, Bojanowski P, et al. Bag of tricks for efficient text classifications. *arXiv preprint arXiv:1607.01759*, 2016.
- [53] Bojanowski P, Grave E, Joulin A, et al. Enriching word vectors with subword information. *Transactions of the association for computational linguistics*, 2017, 5: 135-146
- [54] Kingma D P, Ba J. Adam: A method for stochastic optimization. In: *Proceedings of 3rd International Conference on Learning Representations (ICLR, 2015)*. San Diego, CA, USA, 7-9 May. 2015, ICLR Press, 2015: 136-150
- [55] Goldberg D, Nichols D A, Oki B, et al. Using collaborative filtering to weave an information tapestry. *Communications of the ACM*, 1992, 35(12): 61-70
- [56] He X, Liao L, Zhang H, et al. Neural Collaborative Filtering, in: *Proceedings of the 26th International Conference on World Wide Web (WWW)*, Perth, Canada, 3-7 Apr. 2017, ACM, 2017: 173-182
- [57] Deng Z H, Huang L, Wang C D, et al. DeepCF: A Unified Framework of Representation Learning and Matching Function Learning in Recommender System, in: *Proceedings of the AAAI conference on artificial intelligence*, Hawaii, USA, 27 Jan.-1 Feb. 2019, AAAI, 2019: 61-68
- [58] Briand L, Salha-Galvan G, Bendada W, et al. A semi-personalized system for user cold start recommendation on music streaming apps, in: *Proceedings of the 27th*

ACM SIGKDD Conference on Knowledge Discovery & Data Mining, Virtual Event,
Singapore, 14-18 Aug. 2021, ACM, 2021: 2601-2609